

RESEARCH ARTICLE

Data Overlay Mesh in Distributed Clouds Allowing Collaborative Applications

MILOŠ SIMIĆ¹, JOVANA DEDEIĆ², MILAN STOJKOV¹,
AND IVAN PROKIĆ², (Member, IEEE)

¹Department of Computing and Control Engineering, Faculty of Technical Sciences, University of Novi Sad, 21000 Novi Sad, Serbia

²Department of Fundamentals Sciences, Faculty of Technical Sciences, University of Novi Sad, 21000 Novi Sad, Serbia

Corresponding author: Miloš Simić (milos.simic@uns.ac.rs)

This work was Funded by the European Union (Trustworthy And Resilient Decentralised Intelligence For Edge Systems (TaRDIS), 101093006). Also add the exact same sentence at the beginning of the text in Acknowledgment section, prior to Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union.

ABSTRACT In modern architecture, multi-tenancy is the design mode of operation used to reduce the resource costs of system implementation. This architectural decision does not come without additional costs when managing isolation and data privacy between tenants. That is a known challenge in cloud computing that translates to systems closer to the users, namely edge computing and micro clouds. Existing data management solutions, while supporting multi-tenancy, need help to handle the dynamic nature of such environments, limiting their ability to scale and support collaborative applications across different clusters or regions. This paper addresses the limitations of current data management solutions in supporting dynamic organization, configuration, and collaboration in multi-tenant, geo-distributed micro clouds at the edge. By extending the existing virtual clouds model, we introduce a new dataspace layer and protocols for data discovery using the concept of hierarchical namespaces and transforming non-shareable resources, such as files and folders, into collaborative resources. We propose two types of collaborative applications that leverage stored procedures and event triggers for data consumption. Finally, we ensure that the model provides accurate access and redistribution of shareable (CPU, RAM, disk) and non-shareable resources through graph transformation theory, with correctness validated using multiparty session types. That enables more flexible programming for collaborative applications, improving connectivity across distributed storage systems, and allowing resource borrowing between applications within the same organization. This research seeks to bridge the gap between current tools and the demands of modern distributed systems, enabling more robust, adaptable, and collaborative data management solutions.

INDEX TERMS Cloud computing, distributed systems, edge computing, formal specifications, infrastructure as software, platform, namespaces, data mesh, data overlay, resources borrowing, locally first software, collaborative applications.

I. INTRODUCTION

The increasing complexity and diversity of architectures, components, patterns, and users in modern computing environments have made multi-tenancy a favored approach for addressing performance, fault tolerance, scalability, and security requirements [1]. Multi-tenancy enables multiple tenants to share common resources while maintaining isolation, ensuring performance and data privacy [1]. This model

reduces infrastructure costs, simplifies maintenance, and streamlines updates. Efforts to address multi-tenancy in distributed systems have traditionally focused on centralized data management frameworks or regional clusters [2]. In cloud computing, multi-tenancy is well-established, but challenges remain in more constrained environments such as edge computing and micro clouds (μ Cs). Prominent solutions, such as Kubernetes' multi-tenancy models [3], are designed for single-cloud or regionally scoped deployments, which inherently limit their utility in environments spanning multiple edge μ Cs. EdgeNet [4] and KubeEdge [5]

The associate editor coordinating the review of this manuscript and approving it for publication was Nitin Gupta¹.

are Kubernetes-based clusters aimed to bring Kubernetes features close to the edge, which opens possibilities to address these issues. Google Anthos aims to address the challenges of multi-cloud environments but focuses primarily on infrastructure management, often neglecting the need for collaboration in decentralized, μ C-based infrastructures [6]. In μ C setting, the community faces significant resource allocation and utilization challenges, particularly in ensuring fair distribution of resources and maintaining robust isolation between tenants [7], [8], [9]. Effective management of resources, such as CPU, memory, and storage, is crucial to balance cost-effectiveness with performance. Intelligent resource-sharing approaches are essential for optimizing resource utilization, mainly as tenant workloads fluctuate. In these settings, our previous work pinpointed that namespaces play a critical role in enabling multi-tenancy by providing isolation and preventing interference between tenants [9].

In this paper, we address the problem of insufficient support for a dynamic organization, configuration, and collaboration in existing data management solutions that allow multi-tenancy, particularly in geo-distributed μ Cs at the edge, by extending the existing model for virtual clouds (vClouds) and introducing protocols for data discovery in such environments, relying on the concept of namespaces [9]. Current tools and models struggle to enable collaborative applications and data mesh processing beyond a single cluster or region, limiting their effectiveness in dynamic, multi-cloud environments. This limitation hampers the ability to effectively scale collaborative applications or implement data mesh processing, which requires seamless interaction across distributed and heterogeneous infrastructures. vClouds has shown promise, on which we base our approach in order to support scalable and dynamic collaboration on the edge [9], [10], [11]. Our proposed approach is designed to be adaptable to diverse environments. It leverages formal models, i.e., multiparty asynchronous session types and graph transformations, and it extends existing models by introducing dataspace bonded to namespaces. This adaptability allows for seamless data collaboration and resource sharing across multiple clouds [12]. It also enhances the ability to create links across diverse storage systems. It provides more programming flexibility for developing collaborative, locally-first applications in distributed and dynamic environments through two types of communication – using stored procedures and event triggers for data consumption. That reassures us that our approach can be effectively applied in various scenarios.

In summary, this paper makes several key contributions to the field of distributed data management in edge-based, multi-cloud environments:

- We extend the model for virtual clouds (vClouds) on physical infrastructure with dataspace for data collaboration management. This extension defines primitives for cluster collaborators to gain dynamic access or

borrow initially unowned resources, enabling more efficient and flexible utilization of resources across geo-distributed infrastructures, which is crucial for supporting dynamic workloads and fostering innovation in multi-cloud environments.

- We define protocols to dynamically discover and assign access to, by default, non-shareable resources such as files and folders, transforming them into collaborative resources. This capability is pivotal for fostering real-time collaboration within vClouds, as it facilitates seamless resource sharing and interaction across previously isolated entities, thereby unlocking new possibilities for distributed workflows and cross-cluster applications.
- We propose collaborative applications that use stored procedures and event triggers for data consumption. This approach enhances the responsiveness and scalability of data-driven workflows by enabling automated, event-driven interactions, essential for building efficient and reactive systems within highly dynamic and distributed vCloud environments.
- Our extended model ensures accurate shareable (e.g., CPU, RAM, disk) and non-shareable (e.g., files, folders, and data extension of storage isolation primitives) resource access and redistribution through graph transformation theory using the concept of hard and soft links. The proposed protocols are validated for correctness using multiparty session types for modeling. This model guarantees correctness and isolation and establishes a robust foundation for secure and efficient resource management, crucial for enabling scalable and conflict-free collaboration in geo-distributed vCloud environments.

These advancements rely on our following technical contributions:

- **Protocol definitions for soft link creation:** This study defines a protocol for establishing a soft link to another application's file/folder inside its dataspace. This protocol enables data consumption by leveraging the power of rules for graph transformations over namespaces. It extends a standardized framework for managing and manipulating the namespaces effectively defined in [9].
- **Resource borrowing two-phase protocol:** The paper presents a resource borrowing protocol that enables transferring a deleted application's dataspace to another application within the same organization, using graph transformations. This two-phase protocol outlines the procedures for removing an application and selecting between either removing its dataspace or requesting a merger with another application. Additionally, the dataspace from the removed application is transferred to another application, with the necessary coordination among the applications and namespaces concerned to facilitate resource sharing.

- **Graph theory-based transformations for dynamic modifications of namespaces:** This paper defines transformations for dynamic namespace modifications based on graph rewriting theory. These transformations enable adaptive resource allocation and organization alterations. By leveraging this formal approach, we ensure the consistency and integrity of namespace structures, facilitating efficient and flexible resource management.

These contributions collectively strengthen our comprehensive framework for namespace manipulation by introducing novel mechanisms for managing data collaboration. This paper focuses on providing a formal description of the proposed model rather than its implementation.

The rest of the paper is organized as follows: Section II discusses related work. The idea of dataspace extension of the distributed virtual μ C clouds model is presented in Section III. The design of the system that utilizes namespaces protocols applicable to μ C, including their transformations using graphs, is formally presented in Section IV. Section V gives real-world application areas for the proposed contributions. Section VI gives concluding remarks and future directions of our research.

II. RELATED WORK

A. DATA MANAGEMENT SOLUTIONS IN THE INTERCONNECTED SYSTEMS

Amazon DataZone is a data management service designed to streamline the cataloging, discovering, sharing, and governing data stored across Amazon Web Services (AWS), on-premises systems, and third-party sources [13]. This tool enables administrators to manage and regulate data access using fine-grained controls, ensuring users have the appropriate privileges and context. Amazon DataZone facilitates data sharing and access across different user roles and business users within an organization, allowing them to discover, utilize, and collaborate on data to generate insights. Integrated closely with the AWS ecosystem, it delivers data directly to end users while simplifying architecture by combining various data management services into a centralized Data Lake solution [14]. AWS DataZone offers features such as governing data access across organizational boundaries, connecting data workers through shared data and tools to drive business insights, and automating data discovery and cataloging using machine learning principles [13].

Data Mesh is an innovative approach to sourcing, managing, and accessing data for large-scale analytical use cases [15]. This technology is based on the idea that business domains should have the ability to define, access, and control their own data products, facilitating connections between data from different domains within an enterprise [15]. By managing domains autonomously, Data Mesh addresses the challenges of data availability and accessibility [16]. This approach significantly impacts modern interconnected systems and applications traditionally relying on data lake architecture, where data from various sources can become

difficult to interpret over time [17]. In such traditional setups, consumers often consult producers to understand the data.

Kubernetes is an open-source tool [18] that is quickly gaining traction in the enterprise space, with customers seeking the capability to run a more comprehensive array of workloads, including data management [19]. In recent years, Kubernetes has introduced several building blocks for data management, which administrators provision. Data in a Kubernetes cluster can be treated as a resource in a node's local storage or an externally managed volume, such as those provided by cloud solutions [20].

All the solutions mentioned above lack dynamic organization, configuration support for collaborative applications, data mesh, and processing capabilities beyond the region or multi-cluster level for geo-distributed μ Cs at the edge, where applications can borrow resources. Despite these limitations, they are foundational to collaborative, locally-first applications over a data meshing overlay at the edge.

Our research enhances existing models by introducing the concept of dataspace bound to the isolation level of namespaces and applications within them. This extension facilitates data collaboration across multiple clouds, integrating them into cohesive units — a mesh — that enable seamless collaboration between applications at the infrastructure level. Additionally, we enable users to operate across multiple physical and virtual clouds (vClouds) simultaneously, allowing resource borrowing when necessary for future interconnected applications.

B. DATA ORGANISATION IN DIFFERENT RESEARCH AREAS

The Unix operating system (UNIX) introduced the concept of hard and soft links between filenames and the actual data stored in the file system [21], [22], [23], which are fundamental for creating references to files or directories [24].

Soft or symbolic links act as shortcuts to other files or directories. These links can span multiple file systems, allowing them to reference files and directories located on different storage devices or partitions [21]. A unique characteristic of soft links is their ability to point to files or directories that may not exist or be accessible during link creation. If the target is absent when the link is accessed, it becomes a *dangling* link [23]. Soft links have their own set of permissions and ownership, separate from the original file or directory.

Hard links, on the other hand, are references to the underlying data blocks of a file [21], [23]. Unlike soft links, hard links can only be created within the same file system and cannot span different file systems or diverse storage devices and partitions. Deleting a hard link to a file does not remove the data, but the data is only deleted when the last hard link to that file is removed [21].

Relational database management systems (RDBMS) support more extensive programming interactions through two key concepts: (i) stored procedures and (ii) event triggers [25], [26]. Although these concepts may seem similar

initially, they are fundamentally different in their operation. The primary distinction is in how they are invoked. Users explicitly call stored procedures, whereas event triggers are automatically invoked in response to specific events occurring on a table.

Stored procedures provide a powerful mechanism for users to write application logic stored on the database server. They require parameters and may produce a single output value [26]. In contrast, event triggers are not explicitly called but automatically react to events at the row or statement level, regardless of the number of rows affected. Event triggers can be used for data protection and to synchronize different tables [27].

All previously mentioned research and industry tools lack collaborative solutions between multiple entities and applications in dynamic environments, such as distributed clouds. However, they serve as foundational steps toward developing such applications and logic.

In contrast to these existing solutions, our model supports the creation of hard links in multiple data objects across diverse storage devices, partitions, and distributed file systems. Additionally, it allows for the creation of hard links on data directories. Our approach also advances the concepts of stored procedures and event triggers by providing users with greater programming flexibility in developing these elements within distributed and dynamic environments, such as distributed clouds. This enhancement supports the development of more collaborative applications with locally-first execution and mechanisms to prevent resource starvation.

III. DATASPACE OVERLAY

A. DISTRIBUTED AND VIRTUAL DISTRIBUTED CLOUDS

To address the limitations of traditional cloud computing solutions — particularly in high latency, performance, and fault tolerance — attention has shifted towards alternative approaches grounded in the concept of distributed clouds [28]. Distributed clouds decentralize nodes by geographically dispersing them, bringing services and resources closer to end-users [29]. It is a strategy that aligns with the principles of fog computing, which seeks to deliver computing resources at the network's edge [30]. By organizing nodes into clusters, the μ C model emerges as a solution where resources are dynamically scaled to meet fluctuating demands while preserving the key advantages of distributed clouds, simultaneously addressing the shortcomings as mentioned above of the traditional cloud model [10], [30].

Distributed clouds provide greater flexibility by enabling the dynamic organization of resources into disposable micro-clouds that operate near users and handle requests locally. It is a concept that has garnered significant attention due to its potential to extend cloud-like infrastructure to the network edge, facilitating seamless collaboration between cloud and edge environments [9], [10], [11]. However, the original μ C model lacks robust support for multi-tenancy,

a critical feature in cloud computing that allows users to share computing resources on the same physical infrastructure while maintaining logical isolation [1], [31].

Inspired by cloud computing and commonly used tools, we address this limitation through namespace-based isolation mechanisms that resolve multi-tenancy issues [11], [32], [33], [34]. In the context of μ Cs, designing isolation mechanisms that span multiple clouds is essential, given the capability of μ Cs to operate at both the cloud level and across multiple clouds [11]. An effective namespace design ensures that multiple users or teams can share infrastructure resources while maintaining data isolation and mitigating issues such as noisy neighbors [7], [35].

Thus, namespaces in distributed clouds can be conceptualized as mechanisms for sharing and isolating infrastructure resources, enabling multiple users or teams to run their applications simultaneously, thereby supporting various forms of multi-tenancy in distributed cloud environments [9].

Building on namespaces, we introduce a new abstraction layer called virtual distributed μ Cs (vClouds) at the edge of the physical infrastructure, where users can manage workloads across different μ Cs. This layer encapsulates the existing operating system kernel and user spaces, providing logical isolation while leveraging kernel features to ensure physical resource isolation.

B. TOWARDS IDEA OF DATASPACES

Dynamic resource organization into disposable clouds operating near users and serve their requests locally is a promising concept. In this approach, the cloud-like infrastructure brings elements to users at the network edge, facilitating easier collaboration between the cloud and the edge [10], [11].

Applications operating closer to the data [36] give users the unique ability to process, filter, or transform data at its origin, thus minimizing the need for extensive data transfer to the cloud, which can introduce significant latency [37]. Reducing latency is crucial for applications that require real-time processing for proper decision-making [38]. The concept of virtual distributed clouds (vClouds) [9] enhances local data manipulation by allowing flexible and efficient resource utilization in distributed environments.

Aroca et al. observed that small ARM-based devices offer good performance for building servers and clusters, especially considering their performance per Watt ratio [39]. This observation is particularly relevant for applications running on devices that operate closer to data sources. By leveraging collaboration between applications in specific scenarios, we can maximize the potential of these ARM-based devices, enhancing their efficiency and effectiveness in data processing tasks.

To enable direct collaboration between applications, it is essential to establish a robust layer over data sources (such as file systems, databases, etc.) that supports this capability. This layer must define how applications connect to the data and when and how they access their data for

others to discover and collaborate. The protocols outlined in this research ensure that users can collaborate on data sources by establishing connections between them. These protocols act as a bridge between owners, collaborators, and the data sources, facilitating storage isolation, abstraction, borrowing, and collaboration as required. Our framework enables efficient and secure collaboration while maintaining data management and access control flexibility.

Building on the OS model of dividing memory into two distinct spaces [40]—user space (where user components operate) and kernel space (where OS components operate)—and extending from previous research [9], we introduce a namespace layer. This namespace layer is where shared (virtual) resources reside, integrating user and kernel spaces into vClouds.

Taking this concept further, our research introduces a fourth space called **dataspace**, where data resides. We propose using kernel features for resource isolation, user space hosting applications, and dataspace managing data collaboration. Figure 1 illustrates our four-layered model, depicting the participants involved in context switches and the connections between the four spaces. Our model enhances the organization and management of resources and data, facilitating efficient collaboration and management within distributed and dynamic computing environments.

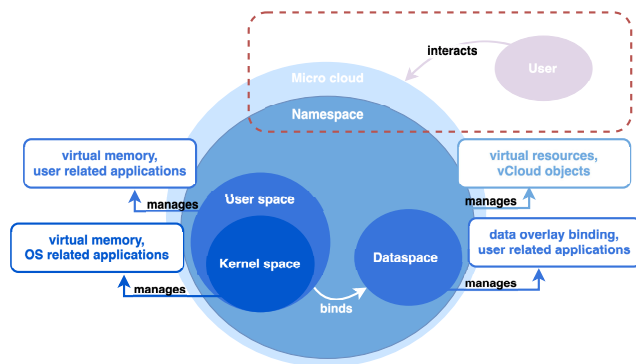


FIGURE 1. Three layered vCloud model with context switch participants, and dataspace.

Compared to existing models where an administrator typically provisions data management, our model offers the capability for dynamic creation and deletion of links to facilitate collaboration (details will be discussed in subsequent sections). In traditional solutions, data are considered a resource within the cluster, residing in node-local storage or managed externally by cloud-provided solutions [20]. These models often rely on pre-defined configurations, offering limited primitives for collaborative data sharing. Moreover, they provide little to no support for flexible borrowing of storage resources between trusted parties, which becomes critical in scenarios where resource requirements fluctuate.

Our model addresses these limitations by incorporating mechanisms for on-demand data sharing and resource borrowing, creating a framework that is not only more

agile but also tailored to dynamic and distributed environments. In addition, it transcends the static and administrator-dependent approaches of prior solutions, providing a more versatile and efficient framework for distributed data management. This innovation supports better collaboration and enables organizations to better adapt to the evolving demands of distributed systems.

The dataspace model enables the binding of diverse policies and connections to the underlying data, reflecting the natural distribution of data within organizations and facilitating collaboration between departments. The dataspace concept represents the entirety of an organization's data collection. Simultaneously, it allows for granular restrictions and enhanced security through permissions applied to each element within the dataspace.

Similarly to how Linux provides permissions for files, folders, and executables [21], our model applies permissions at a similar level of granularity within the dataspace. This approach ensures that data within the dataspace is managed securely and accessibly, aligning with organizational policies and requirements for collaboration while maintaining robust security measures.

Drawing influence from the UNIX *everything is a file* model (i.e., everything has a pathname and is exposed in the file system with that name), the dataspace model recognizes two primary data elements: (i) file and (ii) folder. A file represents a traditional file stored on the disk. In dataspace, a file can also refer to an element within a higher-level data structure, such as a row in a table, an entire column, or any discrete data entity. A folder, similar to traditional file systems, in a dataspace represents a directory where files are organized. Additionally, in a dataspace, a folder can represent a higher-level data grouping, such as an entire table, a collection of related data, or any arbitrary grouping of data that cannot be represented in other ways. By incorporating these concepts into the dataspace model, we enable a flexible and intuitive way to organize and manage data. Our model accommodates traditional file system structures and higher-level data abstractions commonly found in complex data environments, supporting diverse data management needs across various organizational contexts.

To facilitate collaboration within the dataspace overlay, each resource is viewed as a **collaborative resource**. Each resource can be made accessible for other services and users to collaborate on. Certain features and properties are essential for each collaborative resource to support effective collaboration.

C. COLLABORATIVE RESOURCE MESH

Our previous work on vClouds [9] introduced the concept of isolating physical infrastructure resources by utilizing namespaces. This paper proposes a more fine-grained model that categorizes resources into two groups: (i) shareable resources, which include CPU, RAM, disk space, and other resources that can be dynamically allocated and shared among multiple tenants or applications within the

distributed cloud model; (ii) non-shareable resources, which encompasses files, folders, and other data entities that extend the concept of storage isolation primitives.

The resource groups defined in our model can be leveraged by applications running within defined namespaces in a fine-grained manner, offering capabilities that are not easily achievable with existing models. This extension gives our model the unique ability to bind data to namespaces as required.

Our model enables local-first software principles, facilitating easier data management, synchronization, and utilization among applications sharing a namespace. Binding data to namespaces dynamically supports seamless **collaboration** between applications, enhancing agility and efficiency in distributed and dynamic computing environments.

The objective is to tackle the challenges associated with managing non-shareable resources effectively, particularly by developing efficient resource management techniques that facilitate allocating and utilizing these resources within namespaces. To achieve this goal, we must design mechanisms that allow for dynamic assignment of exclusive access to specific resources within namespaces. This involves developing protocols for dynamic resource sharing and coordination between namespaces, thereby transforming non-shareable resources into collaborative resources.

The concept behind collaborative resources is to enable namespaces to share and directly use these resources while imposing restrictions on who, when, and how collaboration can occur. Our model aims to create a more intuitive and natural resource usage experience akin to how companies organize and utilize resources. The new abstraction enables efficient utilization of non-shareable resources, enhancing the flexibility and scalability of namespaces in multi-tenant environments. By allowing dynamic allocation and controlled sharing of resources, our model supports seamless collaboration between applications and tenants while ensuring robust resource management and security.

We must establish an overlay mechanism atop any data storage system (e.g., databases, file systems, etc.) to facilitate smooth operation. This overlay will seamlessly connect non-shareable resources with namespaces and applications. Examining existing systems, we can observe that the Linux file system (FS) addresses similar challenges through its mechanisms.

In the Linux file system, two types of links play crucial roles: (i) *soft link* (symbolic link) and (ii) *hard link*. The soft links serve as simple shortcuts to files or folders [24], [41] and not the content. Soft links are commonly used to link libraries, log files, and directories. They can span different file systems, including Network File System (NFS). The hard links serve as direct references to the original file's data blocks and behave as copies of the original file. They cannot link to directories, and deleting the original file does not affect them [24], [41] until all hard links to the file are removed. Hard links cannot span across NFS.

These ideas are powerful but restrictive in some manner, especially if we want to work in distributed environments. This opens the door for improvements, adjustments, and extensions. Influenced by Linux FS, we envision the same two links but extend them to work in the new complex environments. In our model, we define a **soft link** as a link to the non-shareable resource in the vCloud, in which the application that created this type of link can only read data and never change it. Since the application can only read the data, we can associate specific function(s) on the link, allowing filtering, transformation, and other actions on the data. In some instances, these data manipulations are performed directly in the places where data resides. With these properties, soft links allow local first applications and simultaneously leverage the data locality concept (i.e., moving the computation closer to the data instead of moving extensive data to computation) [42]. One collaborative resource can have **multiple soft links pointing to them**. On the other hand, in the vCloud, the hard link connects the application with the data it created. The hard link confirms the application is the owner of its data. We can transfer ownership to another application if the application is terminated and data needs to be saved instead of counting links. Collaborative resources can have **only one hard link pointing to them**.

Compared to the existing research, our model extends traditional links present in the Linux FS in new directions. It allows creating data mesh between applications and transformation of non-shareable resources into **collaborative resources**, and applications into **collaborative applications**, if and when needed. On the other hand, our model resolves a potential problem that Linux FS has of preserving soft links even if the original file is deleted. In our model, the original collaborative resource can be deleted only if all soft links towards it are deleted first, and applications will be notified about this event. This innovation enhances flexibility, collaboration, and transparency, enabling on-demand resource sharing, eliminating broken links, and integrating applications into the resource lifecycle. By addressing key gaps, our model establishes a robust framework for collaborative resource management, advancing the capabilities of Linux FS for modern distributed systems.

An interesting consequence of our presented model is the possibility of adding soft links on soft links, creating chains or pipelines of data processing, similar to Big Data applications [43], [44]. Our model goes even further, allowing these pipelines to operate collaboratively, enabling users to introduce advanced patterns on collaborative resources close to the data sources (cf. Section III-E).

D. DATASPACE FEATURES AND PROPERTIES

The previous section introduced two types of links: (i) hard links and (ii) soft links. These two types of links, in a sense, give some level of permission for collaborative resources. The concept of permissions is crucial in a multi-tenant environment. Sharing access to data poses a considerable risk

of exposing classified information, data corruption, or even data loss [45].

UNIX added the file permission feature to address these issues in the multi-user environment [22], [45], allowing users to specify how much power others have over a given file or directory [22]. Using specific notation, users can give certain permissions such as: *read* (*r*), *write* (*w*), *execute* (*x*), and *no permission* (-) for three types of users: *owner*, *group*, and *other* [22], [23]. Regarding data types, permissions in UNIX OS include: *files* (-), *folders* (*d*), and *links* (*l*). For instance, using the UNIX permissions notation, record *-rwxrwxrwx* denotes a file (- sign). Then, in groupings of three, we have permissions for owner, group, and others. In our record, the first three symbols represent that the *user creator* (*owner*) has *read*, *write*, and *execute* permissions set. The following three characters represent that *group* have the same permissions set. The last three characters represent that *others* have *read* and *write* permissions set, while *execute* permission is *not set* (- sign).

We use this familiar and commonly used permissions notation adapted here for dataspace and collaborative resources under dataspace. We map the types of users (owner, group, other) at the geo-distribution level. The owner in our model is the application that created the dataspace and its collaborative resources. UNIX group in our model is mapped to a single namespace. Hence, all applications running in the same namespace form a group, giving them group permissions over a dataspace bonded in that namespace. Taking inspiration from astronomy, we use the term **local group**. Applications running in the same namespace where dataspace is bonded can potentially have more freedom than others, e.g., giving more quotas for collaborative resource reading, processing data on the node where it is stored, etc. Other users are those applications residing outside the local group.

In our model, the permissions for users are directly connected with the links (described in the previous section). The owner has exclusive permission to write, which is represented with the hard link. Hence, we write hard link (*h*) for the write permission. All users can have permission to read and execute function(s) on collaborative resources, but some resources can be **undiscoverable** for non-owner users (cf. Section III-E). If a collaborative resource is discoverable, meaning it is open for establishing soft links, we assign permission(s).

Compared to the static nature of UNIX permissions, which define maximum permissions for the owner, group, and others as *rwx* (read, write, execute), our model introduces an enhanced scheme. Here, the owner's maximum permissions are *rhx* (read, hard link, execute), while the local group and others have *rsx* (read, soft link, execute). This distinction fundamentally improves how permissions are managed: (i) Unlike UNIX, where permissions are predefined and rigid, our model allows for on-demand creation of soft links, providing dynamic collaboration without compromising

security; (ii) The separation of hard and soft links offers a clear boundary between ownership and shared access, ensuring greater control over collaborative workflows; (iii) By supporting undiscoverable resources, our model introduces an additional layer of access control that UNIX lacks, allowing resources to remain invisible unless explicitly made discoverable by the owner. These improvements reflect a paradigm shift in permission management, enabling more flexible, secure, and collaborative interactions compared to UNIX. This novel approach addresses the evolving demands of distributed environments where static permission systems fall short.

For example, using our extended UNIX permissions notation, record *-rhxrwxr--* denotes a collaborative resource of type *file* (- sign). Following the groupings of three, the first three characters represent the *owner* of the *file*, with *read*, *hard link* (*write*), and *execute* permissions. The following three characters represent that *local group* (applications inside the same namespace) have *read*, *soft link*, and *execute* permissions set. With execute permission, we allow local group users with soft links to activate their applications when reading data to do some processing or query locally (details in Section III-E). The last three characters say that *others* (applications outside the namespace) have permission to read but are not discoverable and cannot execute the function(s) on a file. Notice that since the collaborative resource is not discoverable (it appears as **hidden**), other users, even though eligible to read, will not be able to use this permission.

To cope with geo-distributed environments, our vCloud model also extends the UNIX three types of users with a *fourth* permission level that sits after the group and before the other part of the user hierarchy. Taking inspiration from astronomy, we use the term **supercluster**. More specifically, a user (application) is in the supercluster of a collaborative resource if they reside in two different namespaces having a common parent namespace. The supercluster level follows the same rules as the group and other permission levels. The idea behind this is simple: give more fine-grained control over resources in a setup where namespaces have a common parent.

Figure 2 shows the permission groups and their jurisdiction in the vCloud model using dataspace.

In Figure 2, each node labeled with *NS* represents the single namespace in the system. The red square represents **owner** level. The blue square represents **local group** level, the green square represents **supercluster** level, while the purple square represents **other** level of permission over the collaborative resources of the dataspace (DS).

We remark that when the soft link and execute permissions are set, we allow the local group or other users to discover the data [46] and collaborate on it. However, setting the permissions might not be enough, since their collaboration is missing when they do not know how data is organized and stored. For data discovery protocol (see Section III-C) to be efficient, besides opening collaborative resources, users

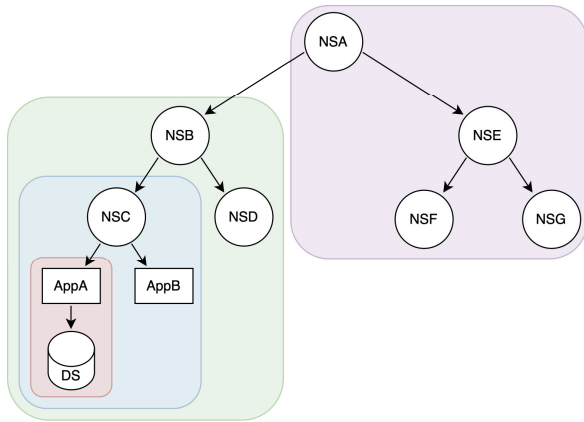


FIGURE 2. Four dataspace permission levels.

must provide **data schema** for data they want to open to others. With this, data owners must specify how data is organized, what the data types are, the missing values, etc. With schema provided and disprovable parameter set, others can then discover data shards, attach to them, and collaborate when needed, creating **collaborative applications or mesh**.

Collaborative resource mesh not only works on data origin and data consumer level, but soft links allow us to create multiple soft links between applications, creating a chain or data pipeline. Every application performs its actions along the way, resulting in multiple actors benefiting and getting data in the best shape and form they need. Creating multiple links allows our model to introduce some of the advanced concurrent processing and communication patterns [47]: (i) fan in, and (ii) fan out shown in Figure 3. Notice dashed lines for some data in the figure, representing data applications linked to it. This means that data is not in their domain and that they have representation of original data **after** application is made its processing, filtering, or any other action.

Since the mesh introduces complex data and their representations obtained through soft links, it is beneficial to talk about their permissions using our extended notation. Consider, for instance, the data of App A in Figure 3 (left) has permissions set `-rhxr-sr--` (as in the previous example). Then, the data of App B (in dashed lines), being a processed representation of App A's data, can have permissions set `lrhxr-sr--`. Notice that the only difference between the two permission sets is the first element: the original App A's data is a file (sign `-`), while the App B's data is a link (sign `l`), i.e., a shortcut to the (processed) original file. Similarly, we can assign permissions to App C and App D data (also in dashed lines), starting with the `l` sign and denoting that these are shortcuts to the (twice processed) original file.

E. COLLABORATIVE APPLICATIONS

The soft links to non-shareable resources in the vCloud allow users to read and execute operation(s) on data. In our setting,

users can attach specific functions on both ends of the link, allowing particular actions on the data. This allows more space for improvements and optimization than the traditional idea of soft links. We envision two types of these function(s): (i) stored procedures and (ii) event triggers.

Stored procedures are simple user-defined functions initiated by their unique identifier (e.g., name, uuid, etc.) and run at the soft link end where data is stored. We can imagine a call over a network where users can provide specific parameters, e.g., procedure name to be initiated and list of arguments. A soft link can have the same name as the stored procedure, meaning that calling the soft link will trigger the corresponding stored procedure. The typical use case of this mechanism is machine learning applications where one application processes, stores, and filters data while another application prepares the data set for training. Since multiple machine learning models can use one data set, stored procedures allow each model to fine-tune preprocessing on the data side but without changing the data.

One caveat with this approach is that the application requiring the data can call stored procedures many times. As a result, the data might not be accessible for other users (i.e., noisy neighbor problem [7]), or requests can overload the node where the data resides, jeopardizing its operability. The resolution of this problem includes: (i) request access limiting [48], [49], and (ii) limiting the range of stored procedure soft links to local group and supercluster.

Regarding access limiting, we rely on the adapted version of the circuit breaker pattern widely used in modern microservice-based applications [49], shown in Figure 4.

When applications reach the previously defined upper bound for accessing the stored procedure. For instance, if excessive attempts are made within a specific time frame, or if excessive data usage strains stored procedure resources and degrades performance, access is denied. This scenario is represented as the *violation* branch in Figure 4. This rate limit violation transitions the application to the *Communication closed* state. The nodes can save this information, collect it as part of metrics, and use it for future analysis. Limiting the range of stored procedures soft links to local group and supercluster is essential because stored procedures **must** guarantee data locality. Our model manages this through the proper graph transformation rules (see Section IV-C).

Besides stored procedures, we have event triggers. In our model, event triggers soft links do not guarantee data locality, for which they are not limited to local group and supercluster, but can also be established for other. This type of soft link operates in two parts. First, we have **trigger** part, which is on the data side. For the trigger, users must provide logic for how and when to query the stored data. On the other (user) side of the link, we have **event** part. This is the side where the data will be delivered, and the event will react upon its arrival. The trigger and event operate asynchronously, creating a communication channel between the sides. This allows more than one event to be connected to the same trigger.

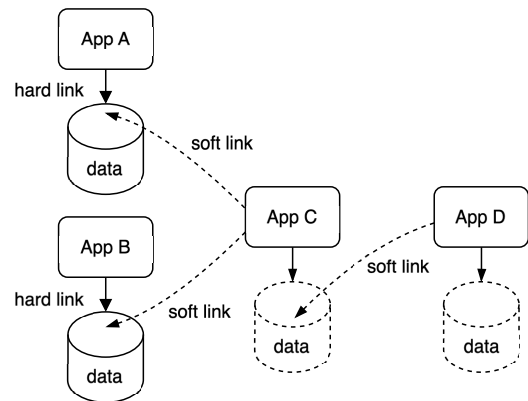
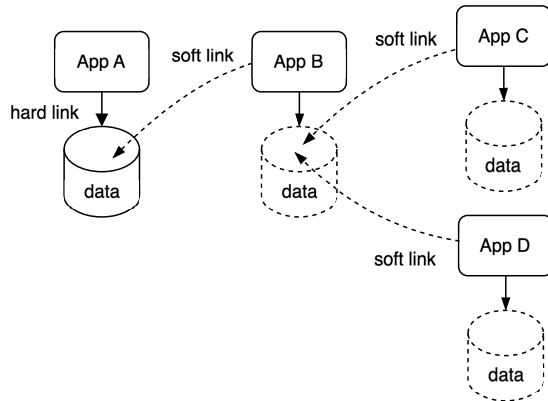


FIGURE 3. Mesh patterns fan in (left), and fan out (right).

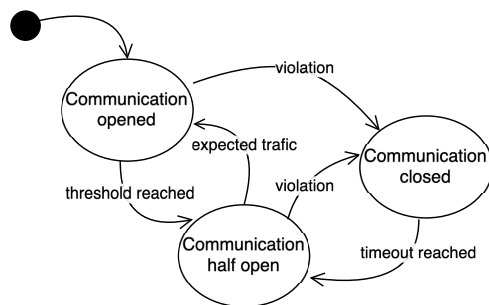


FIGURE 4. Communication limiting mechanisms for stored procedure soft links.

A typical example of when this type of link can be used is when one company governs the data and other companies, teams, and/or users want to use the data or only a specific part of it. For example, a company may run sensors collecting data about air pollution, while hospitals and individual users can get the data in different forms and react upon its arrival.

One question remains: How do users know where the data is located in distributed vCloud and how can they collaborate with it? To answer this question, applications providing data must open the entire data collection, or some specific part of it, for others to locate and use. Therefore, the system must provide the **data discovery protocol**.

Applications opening their data must specify a few essential elements: (i) which elements are opened, (ii) data schema specifying the format of the opened data, (iii) rules for accessing the data (e.g., how often the data can be accessed, how intensive processing the node allows, etc.), and (iv) labels, i.e., a list of key-value pairs that describe opened data. Based on these elements, a namespace with its applications creates the **namespace collaboration tree** so that others can discover its data and collaborate. This represents the first part of the data discovery protocol.

The second part of the data discovery protocol includes applications that discover all namespace collaboration trees. Based on the various options (namespace name, data type,

format, labels, etc.), applications can discover data from other applications, initiate soft link creations, and start collaborations.

F. RESOURCE BORROWING

Section III-C mentioned that a hard link connects the application with the dataspace it created. This indicates that the application is the **owner** of the data and that any data can only have one owner.

However, since we are building a collaborative system, there is an interesting case of what happens with the data of an application when the application is deleted. Here, we recognize two different strategies for the dataspace: (i) a trivial cascade deletion or (ii) re-link the dataspace to some other application. In the first strategy, when the application is terminated and deleted, we also delete its dataspace. In the second strategy, we want to re-link the dataspace to another application, but immediately, we might face challenges for the application: (i) it already has a hard link to its dataspace, and (ii) it does not have enough resources to complete the job.

Let us explore these challenges. The first problem is trivial but should be explored since it always happens in practice. Let us consider a company with two departments, A and B, each department having some local resources. If we want to merge them or move people from department B to department A, for example, but all data and artifacts from B should be transferred to A, we do a simple merge. In this case, the hard link is broken, but the data is transferred. This leads us to the second problem. Suppose the application does not have enough resources to accept the new data. We cannot just merge data because that would break the isolation mechanisms and redistribution of the resources introduced for the namespaces in [9]. Following the redistribution of resources, when the application is terminated, all its resources are **returned** to the namespace, owner of the application. A similar principle is applied if the namespace is terminated, all its resources are returned to its parent namespace [9].

To preserve data locality, the merging of dataspace of the terminated application in our model can be performed

only inside the local group or supercluster. If the merging is performed inside the local group, i.e., the same namespace, then some of the returned resources of the terminated application can be directly given to the merging application if needed. If the merging is performed inside the supercluster, we end up with the second problem mentioned above – the merging application might lack resources. To resolve this problem, the namespace of the application merging the dataspace can **borrow** resources from the namespace of the terminated application. And since on application termination, resources are returned to its owner, we are **guaranteed** that there will be enough resources to complete the task. If, for some reason, the application decides to delete the foreign dataspace, borrowed resources will be returned to the owner namespace.

This scenario supports the locality principle that only known entities (teams, departments, etc.) in the same organization can borrow resources.

For housekeeping purposes, every namespace should record to whom and from whom it borrowed resources. Without this information, the system would not know to whom to return the borrowed resources, therefore we would introduce resource *leaking and hoarding* into the system. All these aspects of borrowing are handled through the graph transformation rule merging dataspace inside the supercluster (see Section IV-C).

One more thing that is important to notice is that a dataspace can have one or more soft links on its collaborative elements, and if two dataspace are to be merged, we must decide what to do with these links. This *conflict* can be quickly resolved by the user merging dataspace using one of two simple rules: (i) mark dataspace as non-collaborative, removing all the links (links may be re-established once the merge is finished) or (ii) preserve the links. Depending on the nature of the merge, the user is in complete control of merging and data collaboration.

IV. SYSTEM DESIGN

A. COMMUNICATION PROTOCOLS

1) MULTIPARTY ASYNCHRONOUS SESSION TYPES

This section gives a brief overview of a formal model [50], an extension of *multiparty asynchronous session types* (MPST) [51], that we use for formal specification and verification of our communication protocols. MPST is a class of behavioral types used to ensure desirable properties of concurrent and distributed communication protocols that rely on asynchronous communication. The model we employ in this work [50] guarantees type safety and progress. Our protocols are validated by the toolchain introduced in [50].

The base of the MPST model is a top-down approach: it provides a formal way of specifying a protocol from the “birds-eye view” perspective, called *global type*. Once the global type is specified, a *projection function* is used to check the correctness and to obtain *local types*, which are local specifications for each of the participants in the session.

The model we utilize in this paper [50], has two important features that extend the standard MPST [51] model: first is a more relaxed form of multiparty choice, and second is optional and dynamic participants.

To provide an intuitive explanation of MPST and to make the paper self-contained, we repeat an example from our previous work [9]. Consider MPST global type:

$$G = \begin{cases} \text{alice} \rightarrow \text{bob} : \text{hello}(\text{int}).G_1 \\ \text{alice} \rightarrow \text{carol} : \text{invite}(\text{nat}) \end{cases} \quad (1)$$

Global type G specifies communication initiated by *alice*: she either sends to *bob* message labeled *hello* with an integer payload, in which case protocol continues as specified by G_1 , or she connects *carol* into the session with sending a message labeled *invite* and a payload of type *nat*, in which case protocol terminates.

Projecting global type G onto participants, we obtain local type specifications. For example, projecting G onto *alice*, *carol*, and *bob* we obtain:

$$S_{\text{alice}} = + \begin{cases} \text{bob}! \text{hello}(\text{int}).S_1 \\ \text{carol}!! \text{invite}(\text{nat}) \end{cases} \quad (2)$$

$$S_{\text{bob}} = \text{alice}? \text{hello}(\text{int}).S_2 \quad (3)$$

$$S_{\text{carol}} = \text{alice}?? \text{invite}(\text{nat}) \quad (4)$$

The above local types give specifications for each participant in the protocol. *alice* chooses between sending to *bob*, in which case continuing as given with S_1 (that is projection of G_1 onto *alice*), or inviting *carol* into the session. *bob* is receiving from *alice* (again, S_2 is projection of G_1 onto *bob*), and *carol* is invited into the session by *alice* and then terminates (assuming *carol* is not involved in G_1).

For more details on session types and their utilization in edge computing, see our previous work [9], [11]. For full details on session types and the model we utilize, see [50].

2) SOFT LINK CREATION

Our first protocol enables an application to establish a soft link towards a collaborative resource of another application. An application gets information on the system’s collaborative resource - file or folder through a discovery service. A soft link is created once initiated and allowed by the application owning the dataspace. In addition to getting access to read-only other application’s data, the data itself can be pre-processed using one of the two options: *stored procedures* or *event triggers*. The protocol relies on the graph transformation rules for soft link manipulation, presented in Section IV-C.

The protocol involves participants: App A, Discovery, Communication Overlay, Registry, App B, and Log, cooperating as shown in Figure 5. We provide the MPST global type specification in (5), as shown at the bottom of the next page.

In the protocol, *App B* initiates the soft link creation for *App A*'s collaborative resource. First, *App B* connects to Communication Overlay with *discoverTree* message and a payload specifying the id of namespace B, typed with T_1 . Discovery servis replies with *resultTree* and the schemes of all opened elements of all applications inside namespace B, typed with T_2 . If the desired tree exists, *App B* *initiate* the soft link creation through a Communication Overlay service also sends the information on where the link is created and if and what stored procedures or event triggers are used, all typed with T_3 . If the tree does not exist, *App B* logs *treeNotFound* message to the Log service, specifying the desired tree in the payload typed with T_{10} . In case the soft link creation has been initiated, the Communication Overlay service connects the collaborative resource owner *App A* with message *allow* and a payload specifying to which collaborative resource soft link will be pointing and who will be pointing (in this case *App B*), all typed with T_4 . Then, *App A* replies with *confirm* message (with an empty payload of the unit type $()$). Now, the Communication Overlay connects the Registry service, informing that soft link creation is *ready*, specifying the application and the collaborative resource to be connected

with a soft link. Also, the payload specifies if a stored procedure or event trigger is to be used and the parameters needed, all typed with T_5 . Then Communication Overlay informs *App B* that the initialization is *done* with a payload of type T_6 . If the stored procedure is used, the Registry connects *App A* with message *pushProcedure* and T_7 being the type of the procedure, after which *App A* replies when the *procedureCreated*, and finally registry informs *App B* that the task is *completed*. In case event triggers are used, Registry connects *App A* with message *pushTrigger*, with the trigger having type T_8 , and also connects *App B* with message *pushEvent* and an event being typed with T_9 . Once finished, *App A* replies with *triggerCreated* and *App B* with *eventCreated*.

Lines (6)-(11), as shown at the bottom of the page, give local type specifications for all of the protocol participants. For instance, line (7) shows that the Discovery service should wait for *App B* to invite them into the session with *discoverTree* message and a payload of type T_1 , and then they should reply with *resultTree* and a payload of type T_2 , after which they disconnect from the session.

$$G_1 = \text{appb} \rightarrow \text{discovery} : \text{discoverTree}(T_1). \text{discovery} \rightarrow \text{appb} : \text{resultTree}(T_2). \\ \left\{ \begin{array}{l} \text{appb} \rightarrow \text{overlay} : \text{intiate}(T_3). \text{overlay} \rightarrow \text{appa} : \text{allow}(T_4). \text{appa} \rightarrow \text{overlay} : \text{confirm}(). \\ \quad \text{overlay} \rightarrow \text{registry} : \text{ready}(T_5). \text{overlay} \rightarrow \text{appb} : \text{done}(T_6). \\ \quad \left\{ \begin{array}{l} \text{registry} \rightarrow \text{appa} : \text{pushProcedure}(T_7). \text{appa} \rightarrow \text{registry} : \text{procedureCreated}(). \\ \quad \text{registry} \rightarrow \text{appb} : \text{completed}() \\ \text{registry} \rightarrow \text{appa} : \text{pushTrigger}(T_8). \text{registry} \rightarrow \text{appb} : \text{pushEvent}(T_9). \\ \quad \text{appa} \rightarrow \text{registry} : \text{triggerCreated}(). \text{appb} \rightarrow \text{registry} : \text{eventCreated}() \end{array} \right. \\ \text{appb} \rightarrow \text{log} : \text{treeNotFound}(T_{10}) \end{array} \right. \quad (5)$$

$$S_{\text{appa}} = \text{overlay} ?? \text{allow}(T_4). \text{overlay} ! \text{confirm}(). \\ + \left\{ \begin{array}{l} \text{registry} ?? \text{pushProcedure}(T_7). \text{registry} ! \text{procedureCreated}() \\ \text{registry} ?? \text{pushTrigger}(T_8). \text{registry} ! \text{triggerCreated}() \end{array} \right. \quad (6)$$

$$S_{\text{discovery}} = \text{appb} ?? \text{discoverTree}(T_1). \text{appb} ! \text{resultTree}(T_2) \quad (7)$$

$$S_{\text{overlay}} = \text{appb} ?? \text{initiate}(T_3). \text{appa} !! \text{allow}(T_4). \text{appa} ? \text{confirm}(). \text{registry} !! \text{ready}(T_5). \text{appb} ! \text{done}(T_6) \quad (8)$$

$$S_{\text{registry}} = \text{overlay} ?? \text{ready}(T_5). \\ + \left\{ \begin{array}{l} \text{appa} !! \text{pushProcedure}(T_7). \text{appa} ? \text{procedureCreated}(). \text{appb} !! \text{completed}() \\ \text{appa} !! \text{pushTrigger}(T_8). \text{appb} !! \text{pushEvent}(T_9). \text{appa} ? \text{triggerCreated}(). \text{appb} ? \text{eventCreated}() \end{array} \right. \quad (9)$$

$$S_{\text{appb}} = \text{discovery} !! \text{discoverTree}(T_1). \text{discovery} ? \text{resultTree}(T_2). \\ + \left\{ \begin{array}{l} \text{overlay} !! \text{initiate}(T_3). \text{overlay} ? \text{done}(T_6). \\ \quad + \left\{ \begin{array}{l} \text{registry} ?? \text{completed}() \\ \text{registry} ?? \text{pushEvent}(T_9). \text{registry} ! \text{eventCreated}() \end{array} \right. \\ \text{log} !! \text{treeNotFound}(T_{10}) \end{array} \right. \quad (10)$$

$$S_{\text{log}} = \text{appb} ?? \text{treeNotFound}(T_{10}) \quad (11)$$

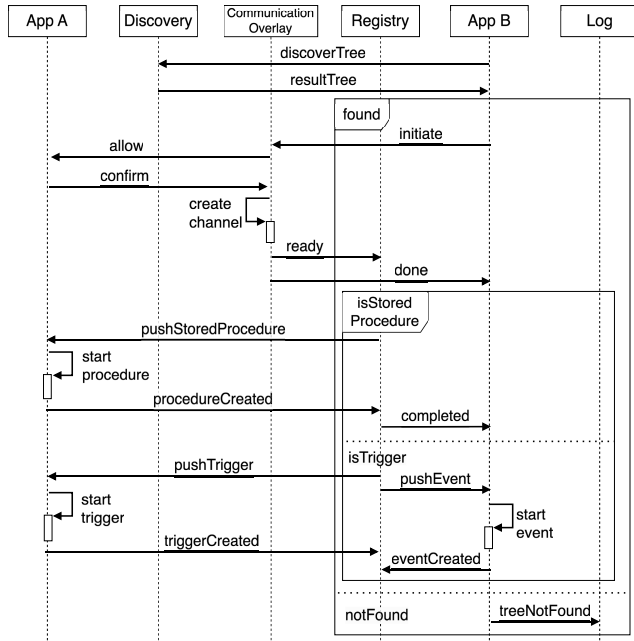


FIGURE 5. Soft link creation protocol.

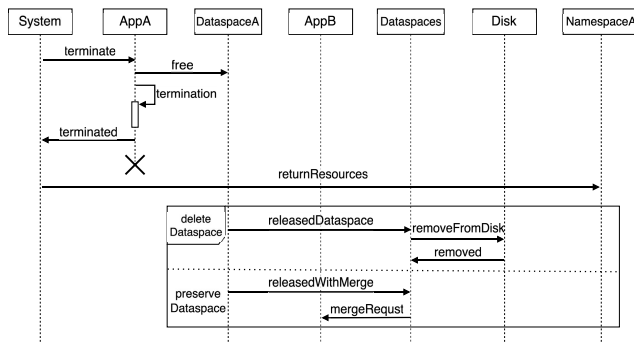


FIGURE 6. Resource borrowing phase 1 protocol.

3) RESOURCE BORROWING PHASE 1 PROTOCOL

Section III-F introduced the concept of resource borrowing that allows assigning the dataspace of an application being deleted to another application. This and the next section show the services involved and their roles in implementing such an operation. We propose a two-phase resource borrowing protocol that aligns with the two phases of the DPO-based graph transformation rules introduced in Section IV-C, which details the resource manipulation in the graph. In the first phase (introduced in this section), the protocol shows the rules for deleting an application and choosing between deleting their dataspace or asking for a merge with another application (see Figure 6). In the second phase (introduced in the next section), the dataspace of the deleted application is assigned to another application, with the proper coordination between applications and namespace(s) involved to achieve resource borrowing (see Figure 7).

The protocol involves participants: System, App A, Dataspace A, App B, Dataspaces, Disk, and Namespace A, cooperating as shown in Figure 6. We provide the MPST global specification in (12), as shown at the bottom of the next page.

The protocol starts with the System service connecting to App A with message *terminate*, specifying if their dataspace (here Dataspace A) is to be deleted or merged with another application, typed with T_1 . Next, the application releases Dataspace A by connecting with message *free* and forwarding deleting/merging decision, typed with T_2 . Then, App A replies to System that they are *terminated* (with the details typed with T_3), which in turn connects Namespace A with message *returnResources*, specifying the amount typed with T_4 . At this point, based on the received information, Dataspace A connects to Dataspaces service with either message *releasedDataspace* with the id of the application and the namespace (typed with T_5) or message *releasedWithMerge* and a payload specifying the details required for merging (such as if the soft links are kept or removed), typed with T_8 . In the former case, Dataspaces connects with Disk with *removeFromDisk*, specifying which dataspace is to be deleted, typed with T_6 , which in turn replies with *removed* and the payload typed with T_7 . In the latter case, Dataspaces connects to App B with *mergeRequest* and a payload of type T_9 , specifying which dataspace will be merged and if the soft links are kept or removed.

Lines (13)-(19), as shown at the bottom of the next page, give local type specifications for all of the participants in the protocol.

4) RESOURCE BORROWING PHASE 2 PROTOCOL

Once phase 1 of the resource borrowing protocol is finished, phase 2 can start. As explained in Section III-F, several different situations must be handled here. If the application merging dataspace is in the same namespace as the deleted application, we check if the application merging has enough resources. If they are in different namespaces, we check if resource borrowing needs to include also the namespace of the deleted application. All these cases are handled with a simple protocol that involves participants: Dataspace A, App B, Dataspaces, Namespace B, and Namespace A, cooperating as shown in Figure 7. We provide the MPST global specification in (20), as shown at the bottom of the page 14.

App B starts the protocol. First, it internally checks its resource restrictions, and if having enough disk space to merge Dataspace A, it starts the subprotocol denoted G' , specified in (21), as shown at the bottom of the page 14. There, App B connects Dataspace A with message *newOwner* and a payload typed with T_1 , after which Dataspace A replies with *doneOwner*, and finally App B connects Dataspaces service informing that the merge is finished with message *doneMerge*, specifying all the details of the merge in a payload typed with T_2 .

If, however, App B does not have enough resources to finish the merging, it connects to their Namespace B with message *requireDisk*, specifying the amount in a payload typed with T_3 . Now, Namespace B has the same internal decision. If it has enough resources, it replies to App B with the message *giveDisk*, and the subprotocol G'_3 again takes place. If Namespace B does not have enough resources, it connects to Namespace A with the message *requireSibiling*, specifying the amount again typed with T_3 .

Then, Namespace A borrows to Namespace B, which gives to App B, and as in all previous cases, G'_3 takes place.

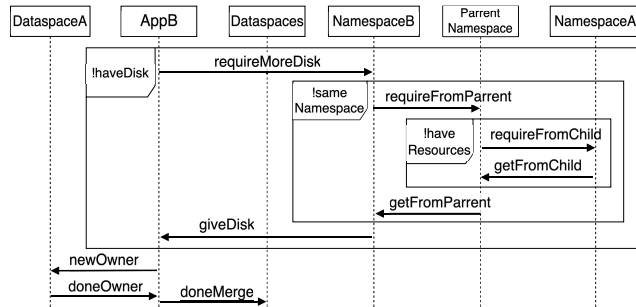


FIGURE 7. Resource borrowing phase 2 protocol.

One important note is that, in the final case, Namespace A must have enough disk space since it previously deleted its App A, which has returned the resources.

Lines (22)–(27), as shown at the bottom of the next page, give local type specifications for all participants in the protocol.

B. FUNDAMENTALS OF NAMESPACES AND DATASPACES

This section formally defines namespaces, applications, and dataspace. Our previous work [9] already gave the definitions of namespaces and applications, which are

here extended to support new definitions of collaborative resources. This extension comes with the price of more details needed, but, on the other hand, allows for some simplifications of the previous definitions.

The organization of namespaces forms a directed acyclic graph (DAG) with a single root element called the default namespace [9]. The namespaces can have multiple applications running inside and each can utilize collaborative resources that can also form a DAG structure. The salient points of this complex structure are presented below.

Namespace is defined as a tuple

$$NS = (C_{ns}, R, B_{in}, B_{out}, A, L, I, S) \quad (28)$$

where:

- C_{ns} is a (possibly empty) set of child namespaces.
- R is a record of the namespace's resources. Formally, $R = \{r_j : v_j\}_{j \in J}$, where $0 \in J$ and J is a finite index set, r_j is a resource label, and v_j is a nonnegative rational number representing the currently available quantity of the resource. Formally, the records in this work are always the mappings from a set of labels (i.e., strings) to the set of nonnegative rational numbers. We reserve r_0 label for the collaborative resource (e.g., hard disk).
- B_{in} is a record of borrowed collaborative resources, i.e., $B_{in} = \{NS_j : v_j\}_{j \in J}$, where J is (possibly empty) finite index set, NS_j is the name of a namespace that borrowed v_j amount of a collaborative resource r_0 to NS . Similarly, $B_{out} = \{NS_j : v_j\}_{j \in J}$ is a record of collaborative resources NS borrowed to other namespaces.
- $A = \{a_j\}_{j \in J}$ is a set of applications a_j (which are described in detail below) running inside the namespace, L is a set of key-value pairs (labels) used for binding and querying, I is general information, and S is the security profile attached to the namespace.

$$G_2 = \text{system} \rightarrow \text{appa} : \text{terminate}(T_1). \text{appa} \rightarrow \text{dsa} : \text{free}(T_2). \text{appa} \rightarrow \text{system} : \text{terminated}(T_3).$$

$$\text{system} \rightarrow \text{nsa} : \text{returnResources}(T_4).$$

$$\begin{cases} \text{dsa} \rightarrow \text{dspaces} : \text{released}(T_5). \text{dspaces} \rightarrow \text{disk} : \text{removeFromDisk}(T_6). \text{disk} \rightarrow \text{dspaces} : \text{removed}(T_7) \\ \text{dsa} \rightarrow \text{dspaces} : \text{releasedWithMerge}(T_8). \text{dspaces} \rightarrow \text{appb} : \text{mergeRequest}(T_9) \end{cases} \quad (12)$$

$$S_{\text{system}} = \text{appa} !! \text{terminate}(T_1). \text{appa} ? \text{terminated}(T_3). \text{nsa} !! \text{returnResources}(T_4) \quad (13)$$

$$S_{\text{appa}} = \text{system} ?? \text{terminate}(T_1). \text{dsa} !! \text{free}(T_2). \text{system} ! \text{terminated}(T_3) \quad (14)$$

$$S_{\text{dsa}} = \text{appa} ?? \text{free}(T_2). + \begin{cases} \text{dspaces} !! \text{released}(T_5) \\ \text{dspaces} !! \text{releasedWithMerge}(T_8) \end{cases} \quad (15)$$

$$S_{\text{appb}} = \text{dspaces} ?? \text{mergeRequest}(T_9) \quad (16)$$

$$S_{\text{dspaces}} = + \begin{cases} \text{dsa} ?? \text{released}(T_5). \text{disk} !! \text{removeFromDisk}(T_6). \text{disk} ? \text{removed}(T_7) \\ \text{dsa} ?? \text{releasedWithMerge}(T_8). \text{appb} !! \text{mergeRequest}(T_9) \end{cases} \quad (17)$$

$$S_{\text{disk}} = \text{dspaces} ?? \text{removeFromDisk}(T_6). \text{dspaces} ! \text{removed}(T_7) \quad (18)$$

$$S_{\text{nsa}} = \text{system} ?? \text{returnResources}(T_4) \quad (19)$$

Application is defined as a tuple

$$a = (R, d, L, I, S) \quad (29)$$

where R is a record of resources the application reserves, and L, I , and S are the application's labels, general information, and security profile. d here represents the application's dataspace, which is defined next.

Dataspace can be thought of as a root folder and is defined as a tuple

$$d = (r_0 : v, F, L, I, S) \quad (30)$$

where L, I , and S are as defined before (but now dataspace specific), v is the currently available quantity of collaborative resource r_0 in the dataspace, $F = \{f_j\}_{j \in J}$ is a (possibly empty) set of files/folders residing directly in the dataspace, defined next.

Collaborative file/folder is defined as a tuple

$$f = (r_0 : v, S_l, F, L, I, S) \quad (31)$$

where F, L, I , and S are again as defined before (but now file/folder specific), v is the reserved quantity of collaborative resource r_0 in the file/folder, and S_l contains information on open/close status of f , and in case of open, also information on stored procedures, event triggers, and soft links pointing to them.

Notice that in namespaces and dataspaces, we denote available resources, while in applications and files/folders, they are reserved. This is a natural consequence of namespaces and dataspaces only representing a grouping, while applications and files/folders represent concrete elements of the system. Notice that compared to our previous work [9], here we go into more detail on applications and introduce dataspaces, files/folders, and resource borrowing. However, notice also that here we simplify the definition of resources for namespaces: in [9] we used pairs of available and utilized,

but here we use only available since the utilized are now specified directly in the applications of the namespace.

The DAG structure makes it natural to use graphs as a graphical representation, which can serve as a basis for defining graph transformation rules that represent dynamic changes in the system. This has been utilized in our previous work [9], where we used multi-weighted directed graphs to represent namespaces: namespaces as nodes, parent-child relation with directed edges, and resource information of namespaces as records assigned to nodes. A naive approach to extend the same representation here leads to a failure: namespaces, applications, dataspaces, and files/folders have different types of resources, while the graphs require unique handling for all nodes and edges. To overcome this limitation, here we use a more general model of **typed graphs** and **typed graph transformations** from [52].

First, we need to identify a **type graph** that defines a set of types for nodes and edges. It also establishes the connections between different types of nodes and edges, while the typing is conducted through a graph morphism between the graph and the identified type graph.

The distinguished type graph for our model is presented in Figure 8. Naturally, it identifies different types of nodes for namespaces, applications, dataspaces, and collaborative files/folders. Each node is assigned the corresponding records of resources (and borrowed resources in the case of namespaces), as described earlier in this section. An important remark is that all namespaces and applications in a single typed graph always have the same set of labels in their resource records. This set of records always contains r_0 , the collaborative resource. In addition, we use one more type for the node representing the open state of the file/folder (cf. S_l information of collaborative file/folder f) - these are not assigned with any labels or records. To make reading easier, we designate different shapes and colors for the nodes of the type graph.

$$G_3 = \begin{cases} G'_3 \\ \text{appb} \rightarrow \text{nsb} : \text{requireDisk}(T_3). \\ \begin{cases} \text{nsb} \rightarrow \text{appb} : \text{giveDisk}().G'_3 \\ \text{nsb} \rightarrow \text{nsa} : \text{requireSibiling}(T_3). \text{nsa} \rightarrow \text{nsb} : \text{getSibiling}(). \text{nsb} \rightarrow \text{appb} : \text{giveDisk}().G'_3 \end{cases} \end{cases} \quad (20)$$

$$G'_3 = \text{appb} \rightarrow \text{dsa} : \text{newOwner}(T_1). \text{dsa} \rightarrow \text{appb} : \text{doneOwner}(). \text{appb} \rightarrow \text{dspaces} : \text{doneMerge}(T_2) \quad (21)$$

$$S_{\text{dsa}} = \text{appb} ?? \text{newOwner}(T_1). \text{appb} ! \text{doneOwner}() \quad (22)$$

$$S_{\text{appb}} = + \begin{cases} S' \\ \text{nsb} !! \text{requireDisk}(T_3). S' \end{cases} \quad (23)$$

$$S' = \text{dsa} !! \text{newOwner}(T_1). \text{dsa} ? \text{doneOwner}(). \text{dspaces} !! \text{doneMerge}(T_2) \quad (24)$$

$$S_{\text{dspaces}} = \text{appb} ?? \text{doneMerge}(T_2) \quad (25)$$

$$S_{\text{nsb}} = \text{appb} ?? \text{requireDisk}(T_3). + \begin{cases} \text{appb} ! \text{giveDisk}() \\ \text{nsa} !! \text{requireSibiling}(T_3). \text{nsa} ? \text{getSibiling}(). \text{appb} ! \text{giveDisk}() \end{cases} \quad (26)$$

$$S_{\text{nsa}} = \text{nsb} ?? \text{requireSibiling}(T_3). \text{nsb} ! \text{getSibiling}() \quad (27)$$

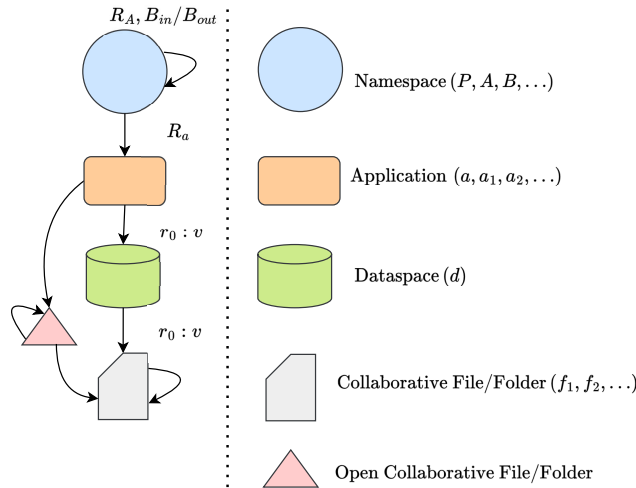


FIGURE 8. Type graph.

Besides the types of nodes, the edges of the type graph determine the structure of all graphs typed with it. As noted earlier, the type graph determines the shape of all typed graphs we are interested in: they all have to be graph morphic with the type graph. We elaborate on all these details in an example typed graph given in Figure 9.

The typed graph in Figure 9 shows a hierarchical namespaces organization [9]: the default namespace P has two children A and D , while A has children B and C . Some namespaces have applications running: B has two b_1 and b_2 , C has c_1 , and D has application d_1 . Each application has the corresponding dataspaces d , while the dataspaces of b_2 application has two collaborative files/folders f_1 and f_2 . Here, f_1 is marked with a loop, meaning it is closed (a soft link cannot be established). f_2 is attached with (two) triangle node(s), meaning it is opened for soft links. Indeed, in the example, two soft links are attached, one from application c_1 , and another from application d_1 . The soft link from c_1 is pointing to the triangle node closer to f_2 , meaning it establishes a stored procedure connection, while the soft link from d_1 is pointing to the second triangle from f_2 , meaning it establishes event triggers connection. The first type of connection can be established only within a local group or supercluster (see transformation rules in the next section) to preserve data locality, while the second does not have this kind of restriction.

Namespaces and applications can have several resources (although they must be assigned with the same set of resources), but in Figure 9 we emphasize only hard disk (HD) since it is the only collaborative resource. For instance, notice that namespace B has 3TB available HD, while their applications b_1 and b_2 reserve 1TB each. Dataspaces of application b_2 specifies that non are available, while their collaborative files/folders f_1 and f_2 reserve 0.4TB and 0.6TB, respectively. Namespace B also specifies that some of their HD resources are borrowed from namespace C (cf. record

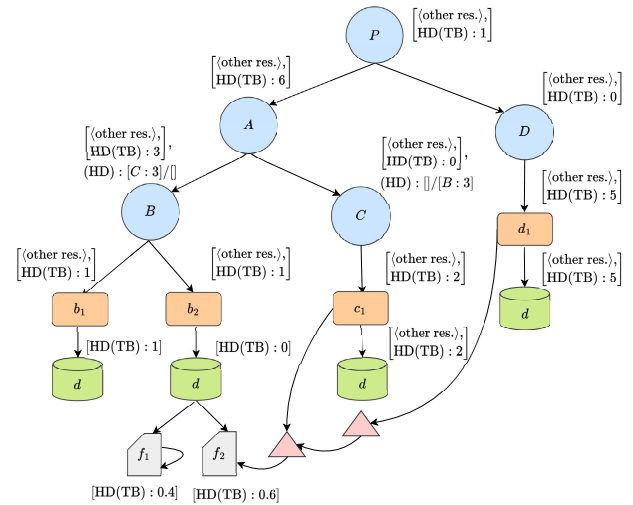


FIGURE 9. Example of resource typed graph.

B_{in} in the definition of NS), concretely here 3TB, while B also specifies that this transfer has happened (in their B_{out} record). The total HD across the organization amounts to 19TB. This information can be extracted using tree search algorithms (since namespace trees are generally small, this should be done fairly quickly).

C. GRAPH-BASED NAMESPACE AND DATASPACE TRANSFORMATIONS

This section presents the rules for dynamic reconfiguring namespaces, applications, dataspaces, and collaborative files/folders. The rules are, as in our previous work [9], based on the double-pushout (DPO) constructs of graph transformations [52]. The graph transformation system in this work extends [9] by introducing transformation rules for the typed graphs to encompass different types of nodes (and not only namespaces) but also simplifies the rules for the namespace manipulation.

The DPO framework defines production rules for graph transformations. A production rule $p = (L \leftarrow K \rightarrow R)$ involves deleting $L \setminus K$ and creating $R \setminus K$ along K . Given p , a graph G , and a graph morphism $m : L \rightarrow G$ (the match), a direct graph transformation $G \Rightarrow H$ can be defined, based on the gluing condition from [52]. Also, a graph transformation system (GTS) comprises a set of production rules. A graph language (GL) for a given G and GTS includes all graphs derivable from G via direct graph transformations.

Our (typed multi-weighted) GTS fundamental rules can be divided into three categories:

- The first category of the rules is for namespace manipulation - these are the rules from our previous work [9], adapted here to encompass resource borrowing and applications - see Section IV-C1.
- The second category of the rules is for application and dataspaces manipulation - see Section IV-C2.

- The third category of the rules is for collaborative files/folders manipulation - see Section IV-C3.

In the next subsections, we use binary operations on records $R_1 - R_2$ and $R_1 + R_2$, which, for two records $R_1 = \{l_i : v_i\}_{i \in I}$ and $R_2 = \{l_j : u_j\}_{j \in J}$, where $J \subseteq I$ (i.e., R_2 having a subset of labels of R_1), are defined as $R_1 - R_2 = R_1 - R'_2$ and $R_1 + R_2 = R_2 + R_1 = R_1 + R'_2$, where

$$R_1 - R'_2 = \begin{cases} \{l_i : (v_i - u_i)\}_{i \in I}, & \forall i \in I : v_i - u_i \geq 0 \\ \text{undefined}, & \text{otherwise} \end{cases}$$

$$R_1 + R'_2 = \{l_i : (v_i + u_i)\}_{i \in I} \quad (32)$$

for $R'_2 = R_2 \cup \{l_i : 0\}_{i \in I \setminus J}$. Hence, the subtraction $R_1 - R_2$ on records is defined as the subtraction of the respected values if the labels of R_2 are also labels of R_1 and R_1 values are greater or equal to the respected values in R_2 . For instance, if $R_1 = \{l_0 : 2\}$, $R_2 = \{l_0 : 1, l_1 : 2\}$, and $R_3 = \{l_0 : 3, l_1 : 2\}$, then $R_1 - R_2$, $R_2 - R_1$, $R_1 - R_3$, and $R_2 - R_3$ are undefined, while $R_3 - R_1 = \{l_0 : 1, l_1 : 2\}$ and $R_3 - R_2 = \{l_0 : 2, l_1 : 0\}$. On the other hand, record addition is defined for any two records in which one set of labels is contained in the other set of labels. For example, for the above defined records R_1 and R_2 , we have $R_1 + R_2 = R_2 + R_1 = \{l_0 : 3, l_1 : 2\}$. We also use the *monus* operation (the truncated subtraction) for non-negative rational numbers, defined as $a \dot{-} b = \max(a - b, 0)$. Finally, we use v_i^h to denote the amount of resource r_i of the h node in the graph (which can be a namespace, application, dataspace, or file/folder).

In the following, we provide a detailed explanation of the production rules required for the DPO approach.

1) PRODUCTION RULES FOR NAMESPACES

a: CREATION OF A NEW NAMESPACE

The rule, presented in Figure 10, outlines that the graph on the left-hand side (a single node P) represents a potential parent namespace, with available resources R_P and borrowed resources B_{in}/B_{out} . After the rule is applied, P in the right-hand side graph has resources $R_P - R_C$, B_{in}/B_{out} (also found in the gluing graph), and it features an associated child namespace with available resources R_C and no borrowed resources $\emptyset/\emptyset$. Notice that the gluing graph (in the middle) imposes minimal conditions for the applicability of the rule: P must have more or equal R_C resources (cf. equation (32)), meaning the child can receive no more than the parent's available resources.

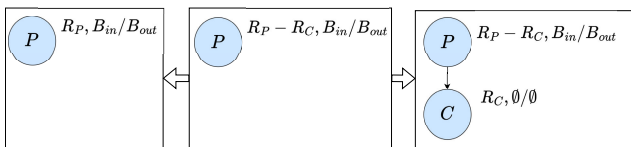


FIGURE 10. Creation of a new namespace.

b: DELETION OF A CHILD NAMESPACE

The deletion of a child namespace is the inverse of the rule for creating a new namespace given in Figure 10, and thus we omit its graphical representation. As noted in [52], the gluing condition of DPO ensures that a rule can only be applied if the deleted child namespace has no children. Additionally, the deleted namespace must be a child, with no resources lent or borrowed. Since no other production rule allows for deleting of namespaces, a default namespace cannot be deleted.

c: RELOCATION OF RESOURCE BETWEEN PARENT-CHILD NAMESPACE

This rule pertains solely to resource manipulation, not namespace transformations. The rule, depicted in Figure 11, illustrates how available resources can be transferred from parent to child namespace and vice-versa with the inverse rule (which we do not present here).

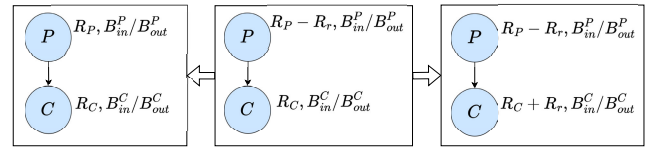


FIGURE 11. Relocation of resources between namespaces.

Notice that our restriction for the resources to be always nonnegative implies the well-definition of the rule, where the parent can transfer at most the amount of available resources.

d: RETURNING THE BORROWED RESOURCES

The rule, presented in Figure 12, specifies two namespaces C and D in the same supercluster (denoted with having common parent P), where C has borrowed v amount of collaborative resource r_0 from D . In the rule, we set $R_b = \{r_0 : v_b\}$, i.e., the record that denotes the borrowed resource. After applying the rule, as specified on the right graph, C has returned the borrowed resource r_0 to D , denoted by C having v less available resource r_0 and removing $C:v$ from the set of borrowed resources (cf. B_{in}^C), and D having v more available resource r_0 and removing $D:v$ from the set of lent resources (cf. B_{out}^D). The gluing graph (in the middle) implies that only if P has available more or equal to v of resource r_0 they can return it to D . Notably, the parent's resources R_P , B_{in}^P/B_{out}^P remain unchanged.

The rule for resource borrowing is presented only in the next section as it involves applications and dataspace.

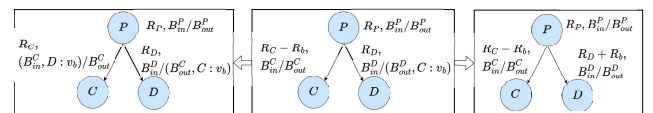


FIGURE 12. Returning the borrowed resources.

2) PRODUCTION RULES FOR APPLICATIONS AND DATASPACE

a: CREATION OF APPLICATION AND THE DATASPACE

The rule, given in Figure 13, specifies that an application and its corresponding dataspace can be created within a namespace. In the namespace before creation, R_P is the amount of available resources and B_{in}/B_{out} are the borrowed resources. After applying the rule (the right graph), a new application a is created with R_a reserved resources dedicated to the namespace. In addition, the dataspace is automatically generated alongside the application via an associated hard link, with the application allocating all its collaborative resources r_0 as available in the dataspace (i.e., in the rule we assume $R_a(r_0)=v_0^a$). The gluing condition implies that an application can be created only with enough resources in the namespace. Notably, the dataspace is established even if its memory capacity is initially empty, i.e., if $v_0^a = 0$.

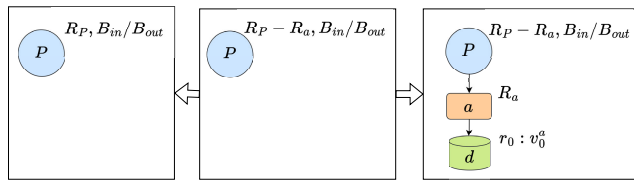


FIGURE 13. Creation of application.

b: DELETION OF APPLICATION

An application in our model can be deleted in three possible ways (cf. Section III-F): (i) the application is deleted together with its dataspace and all the collaborative resources (**cascade deletion**), (1) the application is deleted, but its data space is merged with the dataspace of another application **within the local group**, and (2) same as in the previous case, but the merging is done **within the supercluster**. Next, we present the three rules.

- **Deletion of application - cascade deletion.** This rule is the inverse of the application creation rule shown in Figure 13 - hence, we do not give its graphical representation here. In this rule, the deletion of the dataspace is automatically triggered when the application is deleted. Notice that the gluing condition implies that all the collaborative files/folders must be deleted before the dataspace is deleted (collaborative files/folders are described in the next section).
- **Deletion of application - merging dataspace inside the local group.** The rule, given in Figure 14, specifies that the graph on the left-hand side represents a namespace that contains two applications, a_1 and a_2 , with their data spaces d_1 and d_2 . Application a_2 is deleted and its dataspace d_2 is merged with the dataspace of application a_1 (notice: applications are in the same local group). By already established notation, we have that in d_2 there are $v_0^{d_2}$ available collaborative resource r_0 , while a_2 reserves $v_0^{a_2}$. This implies

$v_t = v_0^{a_2} - v_0^{d_2}$ are the resources taken by the dataspace d_2 that now application a_1 needs to take over. Since d_1 specifies available $v_0^{d_1}$ of r_0 resource, application a_1 requires resource $R_b = \{r_0 : v_t \div v_0^{d_1}\}$ to merge d_2 : if a_1 has more available than d_2 takes we do not need to assign more resources to a_1 , while in the other case we need to assign exactly the difference of the two.

Hence, in the right graph, where application a_2 is deleted, its dataspace d_2 is relinked with the dataspace d_1 of application a_1 . Also, d_2 is marked as having no available resources, as after all its data is merged with d_1 , d_2 will be removed (see the next rule and the rules in the next section). There, all the resources of deleted application a_2 are returned to namespace P , except for those required by application a_1 . Thus, P now has $R_P + R_{a_2} - R_b$ available resources. Furthermore, application a_1 reserves $R_{a_1} + R_b$ resources (i.e., with added required r_0 resources) and its dataspace has $v_0^{d_1} \div v_t$ available of r_0 resource (amount d_1 it had previously subtracted with v_t that d_2 takes, or 0 if the latter is greater than the former).

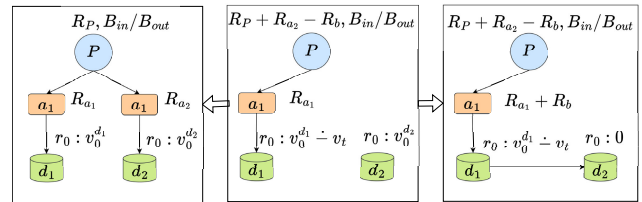


FIGURE 14. Deletion of application - merging dataspace inside the local group.

- **Deletion of application - merging dataspace inside the supercluster.** We emphasize that this rule introduces resource borrowing (as described in Section III-F and in Section IV-A3). The rule, depicted in Figure 15, specifies a similar situation as in the previous case: we delete application a_2 and merge its data to dataspace d_1 of application a_1 . The only difference is that now applications do not share the local group, but the supercluster - application a_1 runs inside namespace C and application a_2 inside D , and C and D have common parent namespace P .

Hence, this rule respects the same transformation as in the previous rule, except for: (i) before borrowing, namespace C gives its r_0 resources to application a_1 - either the missing $v_t \div v_0^{d_1}$ if available in C , or all they have v_0^C , thus we decrease C 's resources for $R = \{r_0 : \min(v_t \div v_0^{d_1}, v_0^C)\}$; (ii) since applications are in different namespaces, we denote if D is borrowing resource r_0 to C (see B_{in}^C and B_{out}^D); (iii) borrowed resources R_b are calculated from resources taken by d_2 subtracted available resources from application a_1 (as in the previous rule) but also the resources available in dataspace C , i.e., $R_b = \{r_0 : v_t \div (v_0^{d_1} + v_0^C)\}$. Hence, resources are borrowed from D to C only if application

a_1 and its dataspace C do not have enough available resources. This is how the borrowed resources are calculated in the Resource borrowing phase 2 protocol in Section IV-A3.

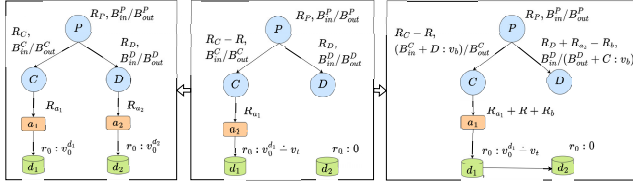


FIGURE 15. Deletion of application - merging dataspace inside the supercluster.

c: DELETION OF DATASPACE

The rule, presented in Figure 16, specifies removing the dataspace (of previously deleted application). The configuration in the left graph arises as a consequence of the application of the previous rule. As a result of applying this rule, we end up with only d_1 , as specified in the right graph. We remark that this rule is applicable only after all files/folders from d_2 are re-connected with d_1 (this transformation is explained in the next section).

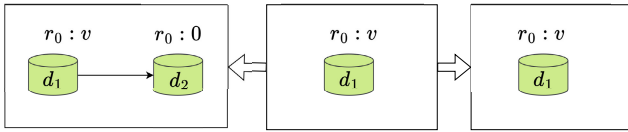


FIGURE 16. Deletion of dataspace.

3) PRODUCTION RULES FOR COLLABORATIVE FILES/FOLDERS

a: CREATION OF COLLABORATIVE FILE/FOLDER

The rule, illustrated in Figure 17, depicts on the left dataspace d with available v_0^d of resource r_0 , while on the right d has created a collaborative file/folder f and assigns to it v_0^f of its resources (where $v_0^d \geq v_0^f$ follows from the definition of typed graphs).

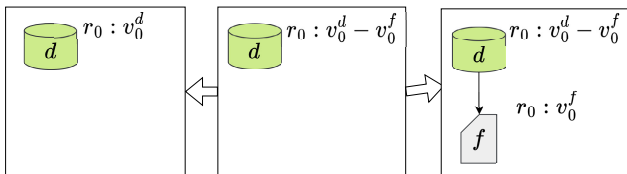


FIGURE 17. Creation of collaborative file/folder.

Deletion of a collaborative file/folder involves the inverse procedure of the one described above (and, by the gluing condition of DPO, is applicable only if f has no subfiles/subfolders).

b: CREATION OF NEW COLLABORATIVE SUBFILE/SUBFOLDER

The rule, presented in Figure 18, illustrates creating f_2 subfile/subfolder (of f_1) in dataspace d , where available resources of dataspace are assigned to new f_2 . Newly created f_2 is decorated with a loop denoting it is closed for soft links and directly connected to d for easier (i) manipulation of resources and (ii) setting the new owner of f_2 (as described in a later rule). The resources of files/folders specify only the size of the file/folder itself. The size of a folder with all its subfiles/subfolders can be reconstructed with a simple tree search algorithm.

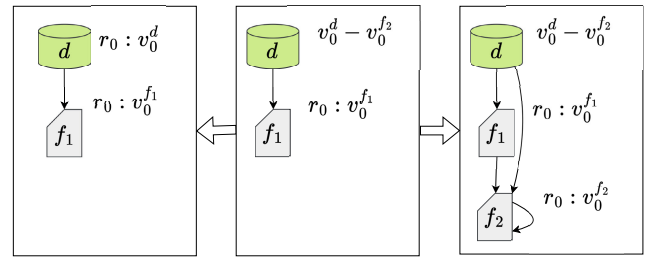


FIGURE 18. Creation of new subfile/subfolder.

Deletion of subfile/subfolder involves the inverse procedure of the one described above.

c: NEW OWNER OF COLLABORATIVE FILE/FOLDER

The rule is presented in Figure 19. The graph on the left-hand side shows that node f is directly linked to the dataspace node d_2 , and is subsequently relinked to d_1 . Consequently, the graph illustrates that when d_2 is bound to d_1 all nodes associated with d_2 including node f_1 , must also be bound to d_1 . This type of node connection change between dataspace and collaborative files/folders is crucial, particularly in the cases of deletion of applications with merging and dataspace deletions (as discussed in the previous sections).

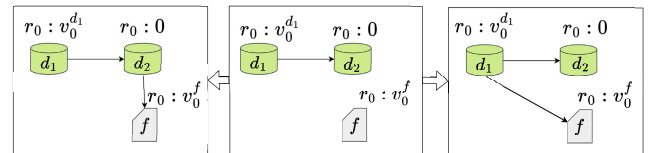


FIGURE 19. New owner of the file/folder.

d: CHANGE THE FILE/FOLDER STATE FROM CLOSED TO OPEN

The rule is presented in Figure 20. The graph on the left-hand side illustrates that node f is initially closed, denoted with a loop, and cannot share data through soft links with the environment. However, on the right-hand side, we observe that the state has changed and node f is now open, allowing it to share information, where through the first triangle we make

stored procedures, and through the second event triggers soft links (discussed below).

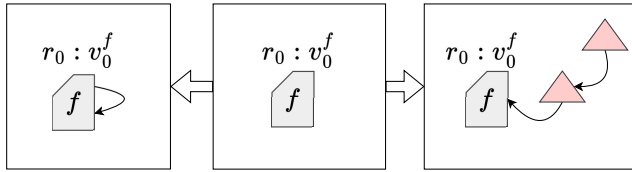


FIGURE 20. Change the state of collaborative file/folder from closed to open.

The rule that describes the **transition from an open to a closed state** is the inverse of the above mentioned rule.

e: CREATION OF SOFT LINKS

As elaborated in Section II-B, soft links represent shortcuts to other files or folders. Thus, we emphasize that no prior redistribution of resources is performed within this rule. We distinguish two types of soft links: **stored procedures** and **event triggers**.

To preserve data locality, stored procedures can be made only inside a **local group** or a **supercluster**. Hence, we have two rules:

- **Creation of stored procedure soft - local group.** The rule is presented in Figure 21. The graph on the left-hand side illustrates two applications a_1 and a_2 running inside namespace P . Application a_1 inside dataspace d_1 has open file/folder f . Node F contains a link with a triangle symbol, indicating that this folder can be used to create a stored procedure soft link. In the graph on the right-hand, from application a_2 , such a stored procedure soft link is created.

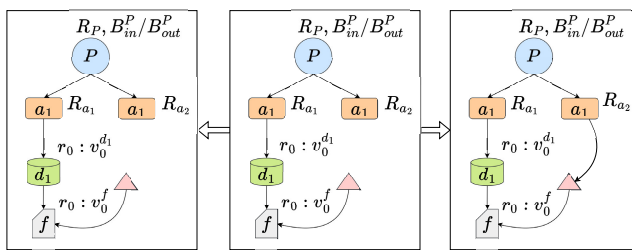


FIGURE 21. Creation of stored procedure soft - local group.

- **Creation of stored procedure soft - supercluster.** The rule is presented in Figure 22. Compared to the previous case, the difference lies in the placement of the hierarchical structure of namespace-type nodes. Here, we have a parent namespace node P with two child nodes, C and D , in which the two applications run. Creating a stored procedure soft link continues in the same manner as in the previous case.

It is important to note that, without delving into the details, **the deletion of a stored procedure** soft link in both cases constitutes the inverse of the abovementioned rules.

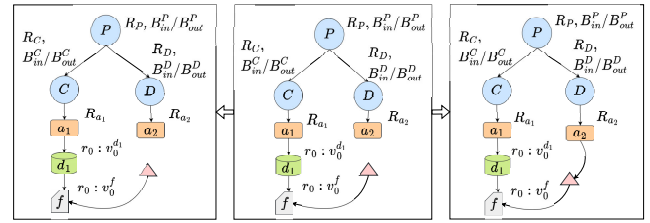


FIGURE 22. Creation of stored procedure soft - supercluster.

f: CREATION OF EVENT TRIGGERS SOFT LINK

The rule is presented in Figure 23. The graph on the left-hand side illustrates that node f is open and can share data with the environment. On the right-hand side, we observe an application a is now connected to f , through an event triggers soft link. Notice that here application a does not have to be neither in the local group nor in the supercluster with the file/folder (we do not require data locality for this type of soft link).

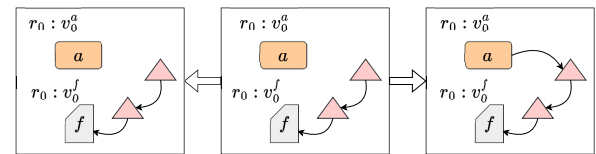


FIGURE 23. Creation of event triggers soft link.

Deleting a event triggers soft link constitutes the inverse of the rule described above.

V. IMPLEMENTATION AND APPLICABILITY

A. PROOF OF CONCEPT IMPLEMENTATION

As discussed in Section I, this paper focuses on theoretical aspects, specifically providing a formal description of the model and its protocols. The protocols aim to establish a collaborative data mesh between applications within a distributed cloud system operating on geo-distributed nodes. While it resembles a traditional cloud, it is designed to operate closer to the users and data sources, making physical infrastructure less rigid than its counterpart. To demonstrate the feasibility of implementing the proposed model, we have developed a proof-of-concept solution as an extension of our previous work [9], [11]. The proposed protocols were tested under controlled laboratory conditions. Figure 24 shows the microservices infrastructure of the proof of concept system.

For service implementation, we used the Go programming language. The popular open-source key-value database *etcd* was utilized as a storage layer due to its support for prefix lookups, which facilitates tree-like storage structures [53]. Additionally, *etcd* allows for asynchronous notifications for critical changes.

Communication between microservices was implemented using remote procedure calls (RPC) via gRPC, with Protobuf

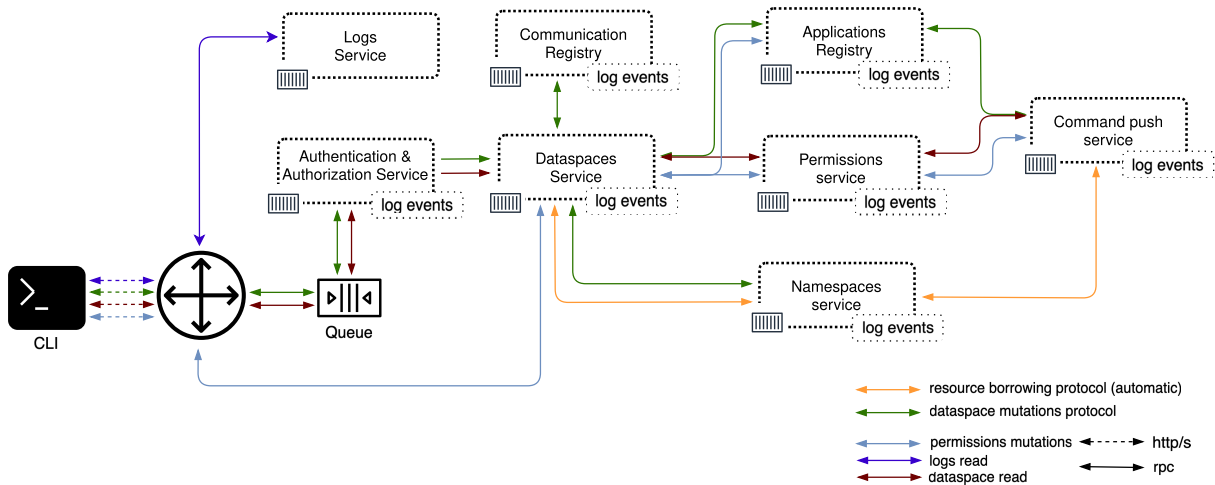


FIGURE 24. Proof of concept system diagram.

used for message definition. These tools were selected for their known scalability, interoperability, and broad applicability, as they are open-sourced [54]. All state changes are described using the YAML format and transmitted to the system gateway via HTTP. We rely on our previous implementation for the delivery of the message to nodes, using NATS as *message oriented middleware*, in a publish-subscribe manner. For handling stored procedures and event triggers, we leveraged Unikraft [55], a fast, secure, and open-source Unikernel Development Kit [56].

The system operates using a set of commands that conform to the defined protocols:

- **Soft link mutations** (represented by green arrows in Figure 24): These commands modify the state of soft links within the system, enabling creation, editing, or deletion operations. When a user specifies the desired operation, its state must be described in a YAML file and sent to the system. It is important to note that for collaborative application creation, unikernel applications must be registered in a designated repository.
- **Resource borrowing** (represented by orange arrows in Figure 24): This is an automated protocol that does not require direct user interaction, except for specifying how to merge the dataspace, which is also defined through YAML. If the system lacks sufficient resources to complete the action, the resource borrowing protocol is triggered automatically, following predefined rules.

Both operations mentioned above are asynchronous by nature, meaning that the system queue accepts user mutation requests without requiring immediate processing. These complex operations require collaboration between multiple services, so users are not expected to wait for their completion to receive feedback. In the event of any issues, a log service (as described in our previous work [11]) is available for users to monitor the state of ongoing operations.

The system also includes several basic operations:

- **Collaborative resource permissions** (represented by purple arrows in Figure 24) allows the creation and reading of collaborative permissions as described in Section III-D.
- **Dataspace read** (represented by brown arrows in Figure 24) retrieves all the necessary data from a single existing dataspace.

Future work will focus on extending the dataspace read operation, particularly for dashboard integration. The extended proof of concept system is capable of creating and maintaining dataspace in previously established distributed vCloud environments. It also manages soft link operations through stored procedure manipulation and event trigger collaboration in applications. Metrics analysis of the implemented system was not conducted, as laboratory conditions differ significantly from real-world scenarios. Evaluating system performance in realistic environments will be a key objective in future work.

The model presented in this research can be applied in two scenarios:

- **Standalone solution:** In this scenario, our previous research [9], [11], combined with the current work, enables the implementation of an entirely new system. It is important to note that developing a new system from scratch may require significant time and effort. Additionally, due to its current level of maturity, cloud providers may be hesitant to adopt such a model at this stage.
- **Extension to existing solutions:** This approach serves as a collaborative data layer, abstracting infrastructure and services to the software level [10], [57]. It leverages the reusability of existing tools, principles, and techniques (e.g., testing, modeling, and evaluation) [58]. This scenario allows cloud providers to integrate the proposed model into their existing infrastructure, create

connectors, and offer collaborative data mesh as a service, utilizing the familiar “pay as you go” model.

As a consequence of the increase in the abstraction level, our model can also be integrated with existing tools (e.g., Kubernetes, Apache Mesos, Docker Swarm, HashiCorp Rancher, etc.) as a *mesh data driver*, a feature currently missing in the existing solutions. This integration provides users with additional flexibility and greater control over how they organize and manage their data workloads.

B. APPLICATION AREAS

Federated Learning (FL) enables data owners to collaboratively train Machine Learning (ML) models without sharing raw data, instead exchanging locally trained private models [59]. For enhanced security, FL can be combined with techniques such as Differential Privacy [60], Multi-Party Computation [61], and Trust Execution Environments [62] for added security. While industrial applications of FL commonly involve training ML models across user devices using local data tailored to specific applications, recent efforts have extended FL to Business-to-Business customers and end-users by performing FL across company servers (i.e., cross-silo FL) on cloud-collected data [63], [64]. Despite substantial progress, several challenges persist for FL, particularly within mobile environments. These challenges include a lack of libraries for background ML training on mobile devices, limited support for cross-application collaboration, and insufficient mechanisms for secure data and model sharing between applications. Additionally, existing FL approaches often do not match the simplicity and ease of use found in ML-as-a-Service platforms such as AWS, Google Cloud, and Azure. To address these challenges, Katevas et al. proposed Federated Learning as a Service (FLaaS), offering solutions in three scenarios [63], [65], [66]:

- 1) **FL models tailored to individual applications** without needing new algorithms, such as a music streaming application modeling user preferences
- 2) **Joint FL models for collaborative applications** solving everyday problems, like image recognition across Facebook, Messenger, and Instagram
- 3) **Joint FL models for novel problems**, such as modeling music preferences based on transportation mode in applications like Uber or Google Maps

The FLaaS ML modeling architecture is a sophisticated system that coordinates client devices that participate in the FLaaS infrastructure. Devices periodically report their status via a REST API, logging the data in a database. An application developer initiates a new FLaaS project through the *Admin Interface*, with the configuration stored in the database. The *Device Scheduler* detects the new project, retrieves device statuses, and sends a training request via a *Notification Service* to external services. Each device receives the request, either collecting local samples for training or initiating model training within third-party applications. The devices then perform on-device training or model

aggregation. Once enough models are reported, the *Device Scheduler* instructs the *Model Aggregator* to combine them and continues with further FL rounds until completion.

Our solution tackles the latter two scenarios, as defined by Katevas et al. [66]. First, in the FLaaS architecture, client devices form vClouds and join namespaces based on locality to be able to send their configuration data to the Device Scheduler, which becomes an integral part of our proposed system. Client devices are not restricted to mobile devices but can be general-purpose computers. Knowing the status of each device, the Device Scheduler can send training requests to client devices with models, which can then use soft links to their non-shareable resources. These soft links allow applications to perform specific actions on the data at both ends, significantly enhancing the potential for optimization and fine-tuning compared to traditional approaches.

For example, stored procedures can process, store, and filter data directly on the client device where it resides, ensuring data locality and reducing unnecessary data movement. This feature is precious in mobile scenarios where multiple ML models may need to operate on the same dataset. The stored procedures enable each model to perform unique filtering without altering the original data. Subsequently, the Model Aggregator collects the individual results, streamlining the process for collaborative model training tailored to solve common problems across various applications within the same organization.

For more heterogeneous scenarios, the event triggers mechanism within FLaaS can provide a powerful solution for cross-organization collaboration. Unlike stored procedures, which operate within a local group or supercluster, event triggers function asynchronously, creating a communication channel between different applications that may belong to separate organizations. To facilitate the use of event triggers, the Device Scheduler needs to be extended to support the data discovery protocol. This configuration enables flexible and secure data interaction, enabling applications to respond in real-time as new data becomes available. By supporting these extensions, FLaaS broadens FL’s applicability across various use cases and fosters innovation and cooperation across industries, ultimately driving more advanced and adaptable machine learning solutions.

VI. CONCLUSION

This paper introduced a model for enhancing collaboration between applications through a new data management layer, the **dataspace**. Our model provides a structured approach to enabling direct collaboration over data owned by different entities with namespace constraints in mind. By defining protocols that govern how applications connect to and manage shared data, our approach facilitates seamless collaboration and efficient resource utilization across different environments.

One of the key contributions of this work is the introduction of the two distinct types of resources — shareable and non-shareable — along with mechanisms for transforming

non-shareable resources into collaborative resources within namespaces. The development of protocols for dynamic resource sharing and coordination enables applications to collaborate more naturally, replicating how organizations organize and use resources in real-world scenarios.

Building on the existing concepts of user space, kernel space, and the previously defined namespace layer, where shared virtual resources reside, we introduced the dataspace layer to handle data collaboration. This layered model, with its connections between spaces, offers a more dynamic and flexible framework for data management, particularly in multi-tenant environments. Unlike traditional models, which typically handle data through administrator-provisioned channels, our approach supports the dynamic creation and deletion of links for collaboration, providing enhanced control and adaptability. Influenced by the design ideas of the Linux file system, we introduced the concepts of soft and hard links within the virtual cloud context as a robust solution for managing collaborative resources. This mechanism ensures that applications can efficiently share and utilize data while maintaining necessary isolation. Creating data processing pipelines through soft links adds further flexibility, enabling advanced collaborative patterns close to the data sources. Moreover, we proposed two mechanisms with rules for communication between collaborative applications - stored procedures and event triggers.

In the final part of the research, we explored potential applicable scenarios where our solution could be applied in federated learning as a service environment. Using FLaaS as a case study, we addressed the complex data management challenges inherent in cross-application Federated Learning scenarios. By leveraging the FLaaS architecture with our proposed solution, we can enable secure and efficient collaboration across applications and organizations, particularly in solving common ML problems and exploring novel use cases. Integrating soft links, stored procedures, and event triggers within the FLaaS framework provides a versatile approach to optimizing and fine-tuning machine learning models. This enhances the practicality of Federated Learning and opens new opportunities for innovation and cross-industry collaboration in the ML domain.

Although our study offers valuable insights, its limitations include the need for more performance evaluation in real-world environments. Future research should replicate this study in practical settings to improve the generalizability of our findings. Another constraint of this work is that deleting data resources requires dismantling all connections to the resources being removed, which may introduce additional complexity in systems with interdependent elements. This constraint could impede efficient resource management, especially in environments with intricate linkages. Furthermore, privacy and security issues are not comprehensively addressed in this work and will be the focus of future research. In addition, future efforts will extend the resource borrowing protocol to support operations between applications within the same namespace and ensure the proper utilization

of fan-in/out patterns to create dynamic and composable computation graphs.

Despite these limitations, our model represents a significant advancement in enabling collaboration between applications and data sources in multi-tenant environments. It offers new possibilities for dynamic resource management, data sharing, and the development of collaborative applications.

ACKNOWLEDGMENT

Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

REFERENCES

- [1] H. AlJahdali, A. Albatli, P. Garraghan, P. Townend, L. Lau, and J. Xu, "Multi-tenancy in cloud computing," in *Proc. IEEE 8th Int. Symp. Service Oriented Syst. Eng.*, Apr. 2014, pp. 344–351.
- [2] H. Ning, Y. Li, F. Shi, and L. T. Yang, "Heterogeneous edge computing open platforms and tools for Internet of Things," *Future Gener. Comput. Syst.*, vol. 106, pp. 67–76, May 2020.
- [3] Kubernetes.io. (2021). *Three Tenancy Models for Kubernetes*. Accessed: Dec. 1, 2024. [Online]. Available: <https://kubernetes.io/blog/2021/04/15/three-tenancy-models-for-kubernetes/>
- [4] B. C. Şenel, M. Mouchet, J. Cappos, O. Fourmaux, T. Friedman, and R. McGeer, "Edgenet: A multi-tenant and multi-provider edge cloud," in *Proc. 4th Int. Workshop Edge Syst., Anal. Netw.*, 2021, pp. 49–54.
- [5] Y. Xiong, Y. Sun, L. Xing, and Y. Huang, "Extend cloud to edge with KubeEdge," in *Proc. IEEE/ACM Symp. Edge Comput. (SEC)*, Oct. 2018, pp. 373–377.
- [6] G. Cloud. (2021). *How Anthos Supports Your Multicloud Needs from Day One*. Accessed: Dec. 1, 2024. [Online]. Available: <https://cloud.google.com/blog/topics/anthos/how-anthos-makes-multicloud-possible-today>
- [7] A. Makroo and D. Dahiya, "A systematic approach to deal with noisy neighbour in cloud infrastructure," *Indian J. Sci. Technol.*, vol. 9, no. 19, pp. 4–7, May 2016, doi: [10.17485/ijst/2016/v9i19/89211](https://doi.org/10.17485/ijst/2016/v9i19/89211).
- [8] E. Ebrahimi, C. J. Lee, O. Mutlu, and Y. N. Patt, "Fairness via source throttling: A configurable and high-performance fairness substrate for multi-core memory systems," *ACM SIGPLAN Notices*, vol. 45, no. 3, pp. 335–346, Mar. 2010.
- [9] M. Simić, J. Dedeić, M. Stojkov, and I. Prokić, "A hierarchical namespace approach for multi-tenancy in distributed clouds," *IEEE Access*, vol. 12, pp. 32597–32617, 2024, doi: [10.1109/ACCESS.2024.3369031](https://doi.org/10.1109/ACCESS.2024.3369031).
- [10] S. Miloš, S. Goran, Z. Miroslav, and M. Branko, "Infrastructure as software in micro clouds at the edge," *Sensors*, vol. 21, no. 21, p. 7001, Oct. 2021, doi: [10.3390/s21217001](https://doi.org/10.3390/s21217001).
- [11] M. Simić, I. Prokić, J. Dedeić, G. Sladić, and B. Milosavljević, "Towards edge computing as a service: Dynamic formation of the micro data-centers," *IEEE Access*, vol. 9, pp. 114468–114484, 2021.
- [12] I. Stoica and S. Shenker, "From cloud computing to sky computing," in *Proc. Workshop Hot Topics Operating Syst.*, Ann Arbor, MI, USA, S. Angel, B. Kasikci, and E. Kohler, Eds., Jun. 2021, pp. 26–32, doi: [10.1145/3458336.3465301](https://doi.org/10.1145/3458336.3465301).
- [13] *How Amazon DataZone Supports and Integrates With Other AWS Services?*. Accessed: Jun. 12, 2024. [Online]. Available: <https://docs.aws.amazon.com/datazone/latest/userguide/what-is-datazone.html>
- [14] R. Raju, R. Mital, and D. Finkelshtein, "Data lake architecture for air traffic management," in *Proc. IEEE/AIAA 37th Digit. Avionics Syst. Conf. (DASC)*, Sep. 2018, pp. 1–6.
- [15] Z. Dehghani and M. Fowler, *Data Mesh: Delivering Data-driven Value at Scale*. Sebastopol, CA, USA: O'Reilly Media, 2022. [Online]. Available: <https://books.google.rs/books?id=M5J5zgEACAAJ>
- [16] J. Majchrzak, S. Balnojan, and M. Siwiak, *Data Mesh in Action*. Shelter Island, NY, USA: Manning, 2023. [Online]. Available: <https://books.google.rs/books?id=DV6mEAAQBAJ>

- [17] R. Hai, C. Quix, and M. Jarke, "Data lake concept and systems: A survey," 2021, *arXiv:2106.09592*.
- [18] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, omega, and kubernetes," *Commun. ACM*, vol. 59, no. 5, pp. 50–57, Apr. 2016, doi: [10.1145/2890784](https://doi.org/10.1145/2890784).
- [19] M. A. Cusack, J. Adamson, M. Brinicombe, N. A. Carson, T. Keijser, J. Peterson, A. Vasudev, K. Westerfeld, and R. Wipfel, "Yellowbrick: An elastic data warehouse on kubernetes," in *Proc. 14th Conf. Innov. Data Syst. Res.*, Cham, Switzerland, HI, USA, 2024, pp. 1–9. [Online]. Available: <https://www.cidrdb.org/cidr2024/papers/p2-cusack.pdf>
- [20] J. Lehmann, S. Schorz, A. Rache, T. Häußermann, M. Rädle, and J. Reichwald, "Establishing reliable research data management by integrating measurement devices utilizing intelligent digital twins," *Sensors*, vol. 23, no. 1, p. 468, Jan. 2023, doi: [10.3390/s23010468](https://doi.org/10.3390/s23010468).
- [21] R. Rosen, *Linux Kernel Networking: Implementation and Theory* (Books for Professionals by Professionals). New York, NY, USA: Apress, 2013. [Online]. Available: <https://books.google.rs/books?id=96V4AgAAQBAJ>
- [22] R. Blum and C. Bresnahan, *Linux Command Line and Shell Scripting Bible*. Hoboken, NJ, USA: Wiley, 2015. [Online]. Available: <https://books.google.rs/books?id=NhAmBgAAQBAJ>
- [23] M. R. Yaswinski, M. M. Chowdhury, and M. Jochen, "Linux security: A survey," in *Proc. IEEE Int. Conf. Electro Inf. Technol. (EIT)*, May 2019, pp. 357–362.
- [24] M. Maleki, "Links and inodes in Linux file system," in *Developers Ultimate Guide: Linux Bash Scripting*, 2022.
- [25] R. Stones and N. Matthew, *Beginning Databases with PostgreSQL: From Novice to Professional* (Books for Professionals by Professionals). New York, NY, USA: Apress, 2006. [Online]. Available: <https://books.google.rs/books?id=Nc5ZT2X5mOcC>
- [26] K. Owens, *Programming Oracle Triggers and Stored Procedures* (Prentice-Hall PTR Oracle Series). Upper Saddle River, NJ, USA: Prentice-Hall, 2004. [Online]. Available: <https://books.google.rs/books?id=xLAxulfhCHUC>
- [27] M. Stonebraker, "Triggers and inference in database systems," in *On Knowledge Base Management Systems: Integrating Artificial Intelligence and Database Technologies* (Topics in Information Systems), M. L. Brodie and J. Mylopoulos, Eds., Islamorada, FL, USA: Springer, 1985, pp. 297–314.
- [28] B. Nair and S. M. S. Bhanu, "Fog-cloud collaboration for real-time streaming applications: FCC for RTSAs," in *Handbook of Research on the IoT, Cloud Computing, and Wireless Network Optimization*. Hershey, PA, USA: IGI Global, 2019, pp. 128–147.
- [29] V. B. C. Souza, W. Ramírez, X. Masip-Bruin, E. Marín-Tordera, G. Ren, and G. Tashakor, "Handling service allocation in combined fog-cloud scenarios," in *Proc. IEEE Int. Conf. Commun. (ICC)*, May 2016, pp. 1–5.
- [30] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st, Ed., MCC Workshop Mobile Cloud Comput.*, 2012, pp. 13–16.
- [31] B. C. Şenel, M. Mouchet, J. Cappos, T. Friedman, O. Fourmaux, and R. McGeer, "Multitenant containers as a service (CaaS) for clouds and edge clouds," *IEEE Access*, vol. 11, pp. 144574–144601, 2023.
- [32] K. Hightower, B. Burns, and J. Beda, *Kubernetes: Up and Running Dive Into the Future of Infrastructure*. Sebastopol, CA, USA: O'Reilly Media, 2017.
- [33] Kubernetes.io. (2020). *Introducing Hierarchical Namespaces*. Accessed: Jun. 10, 2024. [Online]. Available: <https://kubernetes.io/blog/2020/08/14/introducing-hierarchical-namespaces/>
- [34] M. Azure. (2023). *Azure Namespace*. Accessed: Jun. 10, 2024. [Online]. Available: <https://learn.microsoft.com/-us/rest/api/servicebus/resource-provider-apis>
- [35] B. Alouffi, M. Hasnain, A. Alharbi, W. Alosaimi, H. Alyami, and M. Ayaz, "A systematic literature review on cloud computing security: Threats and mitigation strategies," *IEEE Access*, vol. 9, pp. 57792–57807, 2021.
- [36] A. Khune and S. Pasricha, "Mobile network-aware middleware framework for cloud offloading: Using reinforcement learning to make reward-based decisions in smartphone applications," *IEEE Consum. Electron. Mag.*, vol. 8, no. 1, pp. 42–48, Jan. 2019.
- [37] S. Alonso-Monsalve, F. García-Carballeira, and A. Calderón, "A heterogeneous mobile cloud computing model for hybrid clouds," *Future Gener. Comput. Syst.*, vol. 87, pp. 651–666, Oct. 2018, doi: [10.1016/j.future.2018.04.005](https://doi.org/10.1016/j.future.2018.04.005).
- [38] J. Cao, Q. Zhang, and W. Shi, *Edge Computing: A Primer* (SpringerBriefs in Computer Science). Cham, Switzerland: Springer, 2018. [Online]. Available: <https://books.google.rs/books?id=RIZwuwEACAAJ>
- [39] R. V. Aroca and L. M. G. Gonçalves, "Towards green data centers: A comparison of x86 and ARM architectures power efficiency," *J. Parallel Distrib. Comput.*, vol. 72, no. 12, pp. 1770–1780, Dec. 2012, doi: [10.1016/j.jpdc.2012.08.005](https://doi.org/10.1016/j.jpdc.2012.08.005).
- [40] S. S. Murtaza, A. Sultana, A. Hamou-Lhadj, and M. Couture, "On the comparison of user space and kernel space traces in identification of software anomalies," in *Proc. 16th Eur. Conf. Softw. Maintenance Reeng.*, Mar. 2012, pp. 127–136.
- [41] C. Benvenuti, *Understanding Linux Network Internals—Guided Tour to Networking on Linux*. Sebastopol, CA, USA: O'Reilly, 2005. [Online]. Available: <http://www.oreilly.de/catalog/understandlni/index.html>
- [42] Z. Guo, G. Fox, and M. Zhou, "Investigation of data locality in MapReduce," in *Proc. 12th IEEE/ACM Int. Symp. Cluster, Cloud Grid Comput. (CCGrid)*, Ottawa, ON, Canada, May 2012, pp. 419–426, doi: [10.1109/CCGRID.2012.42](https://doi.org/10.1109/CCGRID.2012.42).
- [43] H. Foidl, V. Golendukhina, R. Ramler, and M. Felderer, "Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers," *J. Syst. Softw.*, vol. 207, Jan. 2024, Art. no. 111855, doi: [10.1016/j.jss.2023.111855](https://doi.org/10.1016/j.jss.2023.111855).
- [44] P. O'Donovan, K. Leahy, K. Bruton, and D. T. J. O'Sullivan, "An industrial big data pipeline for data-driven analytics maintenance applications in large-scale smart manufacturing facilities," *J. Big Data*, vol. 2, no. 1, p. 25, Dec. 2015, doi: [10.1186/s40537-015-0034-z](https://doi.org/10.1186/s40537-015-0034-z).
- [45] X. Zhang, N. Wuwong, H. Li, and X. Zhang, "Information security risk management framework for the cloud computing environments," in *Proc. 10th IEEE Int. Conf. Comput. Inf. Technol. (CIT)*, Bradford, U.K., Jun. 2010, pp. 1328–1334, doi: [10.1109/CIT.2010.501](https://doi.org/10.1109/CIT.2010.501).
- [46] C. D. Souza, D. L. Ferreira, and C. A. V. Campos, "A protocol for data discovery and retrieval in content-centric and delay-tolerant networks," in *Proc. IEEE 86th Veh. Technol. Conf. (VTC-Fall)*, Toronto, ON, Canada, Sep. 2017, pp. 1–5, doi: [10.1109/VTCFall.2017.8288209](https://doi.org/10.1109/VTCFall.2017.8288209).
- [47] K. Cox-Buday, *Concurrency in Go: Tools & Techniques for Developers*. Sebastopol, CA, USA: O'Reilly Media, 2017. [Online]. Available: <https://books.google.rs/books?id=qmzqswEACAAJ>
- [48] K. Mathews, C. Kramer, and R. Gotzhein, "Token bucket based traffic shaping and monitoring for WLAN-based control systems," in *Proc. IEEE 28th Annu. Int. Symp. Pers., Indoor, Mobile Radio Commun. (PIMRC)*, Montreal, QC, Canada, Oct. 2017, pp. 1–7, doi: [10.1109/PIMRC.2017.8292201](https://doi.org/10.1109/PIMRC.2017.8292201).
- [49] G. Aquino, R. F. de Queiroz, G. V. Merrett, and B. M. Al-Hashimi, "The circuit breaker pattern targeted to future IoT applications," in *Proc. Int. Conf. Service-Oriented Comput.*, in Lecture Notes in Computer Science, vol. 11895, Toulouse, France, S. Yangui, I. B. Rodriguez, K. Drira, and Z. Tari, Eds., Cham, Switzerland: Springer, 2019, pp. 390–396, doi: [10.1007/978-3-030-33702-5_30](https://doi.org/10.1007/978-3-030-33702-5_30).
- [50] R. Hu and N. Yoshida, "Explicit connection actions in multiparty session types," in *Proc. Int. Conf. Fundam. Approaches Softw. Eng.*, in Lecture Notes in Computer Science, vol. 10202, Uppsala, Sweden, M. Huisman and J. Rubin, Eds., Cham, Switzerland: Springer, Apr. 2017, pp. 116–133, doi: [10.1007/978-3-662-54494-5_7](https://doi.org/10.1007/978-3-662-54494-5_7).
- [51] K. Honda, N. Yoshida, and M. Carbone, "Multiparty asynchronous session types," in *Proc. 35th Annu. ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang.*, San Francisco, CA, USA, G. C. Necula and P. Wadler, Eds., Jan. 2008, pp. 273–284, doi: [10.1145/1328438.1328472](https://doi.org/10.1145/1328438.1328472).
- [52] H. Ehrig, K. Ehrig, U. Prange, and G. Taentzer, *Fundamentals of Algebraic Graph Transformation* (Monographs in Theoretical Computer Science. An EATCS Series). Cham, Switzerland: Springer, 2006.
- [53] D. Belazzougui, P. Boldi, R. Pagh, and S. Vigna, "Fast prefix search in little space, with applications," in *Proc. Eur. Symp. Algorithms*, in Lecture Notes in Computer Science, vol. 6346, M. de Berg and U. Meyer, Eds., Cham, Switzerland: Springer, 2010, pp. 427–438, doi: [10.1007/978-3-642-15775-2_37](https://doi.org/10.1007/978-3-642-15775-2_37).
- [54] M. Bolanowski, K. Zak, A. Paszkiewicz, M. Ganzha, M. Paprzycki, P. Sowinski, I. Lacalle, and C. E. Palau, "Efficiency of REST and grpc realizing communication tasks in microservice-based ecosystems," in *Proc. Int. Conf. New Trends Intell. Softw. Methodol., Tools Techn.*, in Frontiers in Artificial Intelligence and Applications, vol. 355, Kitakyushu, Japan, H. Fujita, Y. Watanobe, and T. Azumi, Eds., 2022, pp. 97–108, doi: [10.3233/FAIA20242](https://doi.org/10.3233/FAIA20242).

- [55] S. Kuenzer, V.-A. Bădoiu, H. Lefevre, S. Santhanam, A. Jung, G. Gain, C. Soldani, C. Lupu, C. Teodorescu, C. Răducanu, C. Banu, L. Mathy, R. Deaconescu, C. Raiciu, and F. Huici, "Unikraft: Fast, specialized unikernels the easy way," in *Proc. 16th Eur. Conf. Comput. Syst.*, New York, NY, USA, Apr. 2021, pp. 376–394, doi: [10.1145/3447786.3456248](https://doi.org/10.1145/3447786.3456248).
- [56] R. Pavlicek, *Unikernels: Beyond Containers to the Next Generation of Cloud*. Sebastopol, CA, USA: O'Reilly Media, 2016. [Online]. Available: <https://books.google.rs/books?id=qfDXuQEACAAJ>
- [57] B. Fitzgerald, N. Forsgren, K.-J. Stol, J. Humble, and B. Doody, "Infrastructure is software too!" *SSRN Electron. J.*, vol. 2015, Oct. 2015, Art. no. 2681904.
- [58] M. Artac, T. Borovssak, E. Di Nitto, M. Guerriero, and D. A. Tamburri, "DevOps: Introducing infrastructure-as-code," in *Proc. IEEE/ACM 39th Int. Conf. Softw. Eng. Companion (ICSE-C)*, May 2017, pp. 497–498.
- [59] J. Konečný, H. Brendan McMahan, F. X. Yu, P. Richtárik, A. Theertha Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," 2016, *arXiv:1610.05492*.
- [60] R. C. Geyer, T. Klein, and M. Nabi, "Differentially private federated learning: A client level perspective," 2017, *arXiv:1712.07557*.
- [61] R. Kanagavelu, Z. Li, J. Samsudin, Y. Yang, F. Yang, R. S. M. Goh, M. Cheah, P. Wiwatphonthana, K. Akkarajitsakul, and S. Wang, "Two-phase multi-party computation enabled privacy-preserving federated learning," in *Proc. 20th IEEE/ACM Int. Symp. Cluster, Cloud Internet Comput. (CCGRID)*, May 2020, pp. 410–419.
- [62] F. Mo, H. Haddadi, K. Katevas, E. Marin, D. Perino, and N. Kourtellis, "PPFL: Privacy-preserving federated learning with trusted execution environments," in *Proc. 19th Annu. Int. Conf. Mobile Syst., Appl., Services*, Jun. 2021, pp. 94–108.
- [63] K. Katevas, D. Perino, and N. Kourtellis, "FLaaS: Cross-app on-device federated learning in mobile environments," 2022, *arXiv:2206.10963*.
- [64] M. Tang and V. W. Wong, "An incentive mechanism for cross-silo federated learning: A public goods perspective," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, May 2021, pp. 1–10.
- [65] N. Kourtellis, K. Katevas, and D. Perino, "FLaaS: Federated learning as a service," in *Proc. 1st Workshop Distrib. Mach. Learn.*, Barcelona, Spain, Dec. 2020, pp. 7–13, doi: [10.1145/3426745.3431337](https://doi.org/10.1145/3426745.3431337).
- [66] K. Katevas, D. Perino, and N. Kourtellis, "FLaaS—practical federated learning as a service for mobile applications," in *Proc. 23rd Annu. Int. Workshop Mobile Comput. Syst. Appl.*, Tempe, AZ, USA, R. LiKamWa and U. Hengartner, Eds., Mar. 2022, p. 130, doi: [10.1145/3508396.3517074](https://doi.org/10.1145/3508396.3517074).



MILOŠ SIMIĆ received the B.Sc., M.Sc., and Ph.D. degrees in computer science from the Faculty of Technical Sciences, University of Novi Sad, in 2014, 2015, and 2022, respectively. He has been an Assistant Professor with the Department of Computing and Control, Faculty of Technical Sciences, University of Novi Sad, since 2022. His research interests include distributed systems, cloud computing, edge computing, big data, and service-oriented architectures.



calculi, type theory, and discrete mathematics.

JOVANA DEDEIĆ received the B.Sc. and M.Sc. degrees in mathematics from the Faculty of Sciences, in 2010 and 2011, respectively, and the Ph.D. degree in applied mathematics from the Faculty of Technical Sciences, University of Novi Sad, in January 2022. She has been an Assistant Professor with the Department of Fundamentals Sciences, Faculty of Technical Sciences, University of Novi Sad, since 2023. Her research pursuits primarily revolve around formal methods, process



MILAN STOJKOV received the B.Sc., M.Sc., and Ph.D. degrees in computer science from the University of Novi Sad, in 2014, 2015, and 2022, respectively. He has been an Assistant Professor with the Department of Computing and Control Engineering, Faculty of Technical Sciences, University of Novi Sad, since 2023. His research focuses primarily on distributed systems security, applied cryptography, critical infrastructures, and software architectures.



methods, process calculi, type theory, and discrete mathematics.

IVAN PROKIĆ (Member, IEEE) received the B.Sc. and M.Sc. degrees in mathematics from the Faculty of Sciences, University of Novi Sad, in 2012 and 2014, respectively, and the Ph.D. degree in applied mathematics from the Faculty of Technical Sciences, University of Novi Sad, in 2020. He has been an Assistant Professor with the Department of Fundamentals Sciences, Faculty of Technical Sciences, University of Novi Sad, since 2021. His research interests include formal

...